



分布式系统

中国人民大学信息学院 · 鄂金龙

本课目录

- 分布式系统概述
- 分布式通信
- 分布式存储
- 分布式计算

本课目录

- **分布式系统概述**
- 分布式通信
- 分布式存储
- 分布式计算

概念与特点

- **什么是分布式系统?**
  - **定义1**：一个分布式系统是一些独立的计算机的集合，但是对这个系统的用户来说系统就像一台计算机一样
    - 从硬件角度：每台计算机都是自主的
    - 从软件角度：对用户来说就像与单个系统打交道
  - **定义2**：一组协同工作的计算机通过网络互相连接，互相传递消息协调它们的行为而形成的系统









概念与特点

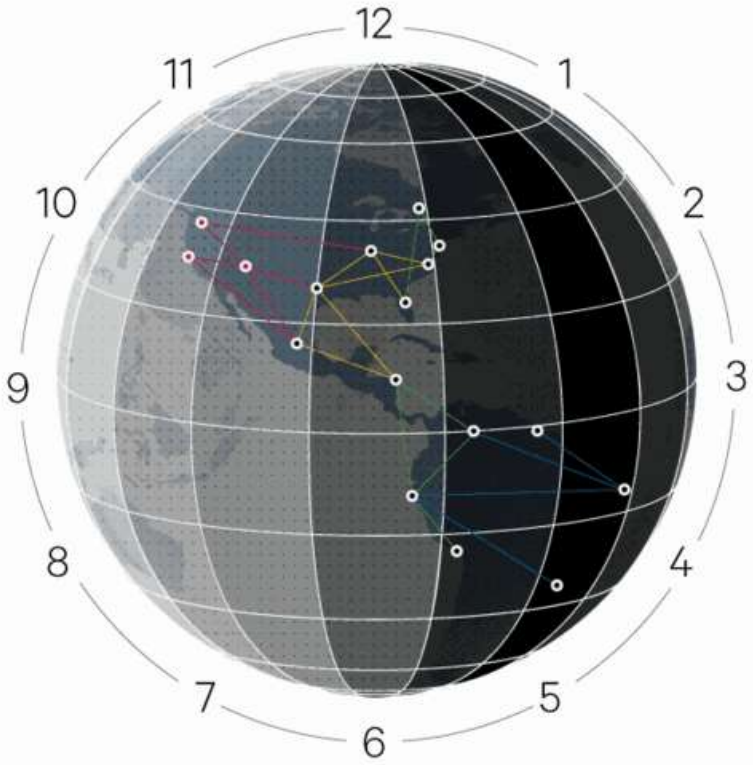
- **分布式**（Distributed）**vs. 并行**（Parallel）
  - **共同点**
    - 都是多机处理任务，重点考虑机器间负载均衡、通信和同步，以处理性能为目标 *（广义上讲分布式包括并行）*
  - **区别**
    - 并行通常处理一个超大任务，强调并行度，追求计算快，效率和成本其次，主要用于超算（Super Computing）
    - 分布式处理多个任务，综合考虑性能/可靠/成本/服务质量等目标，企业级实际应用多





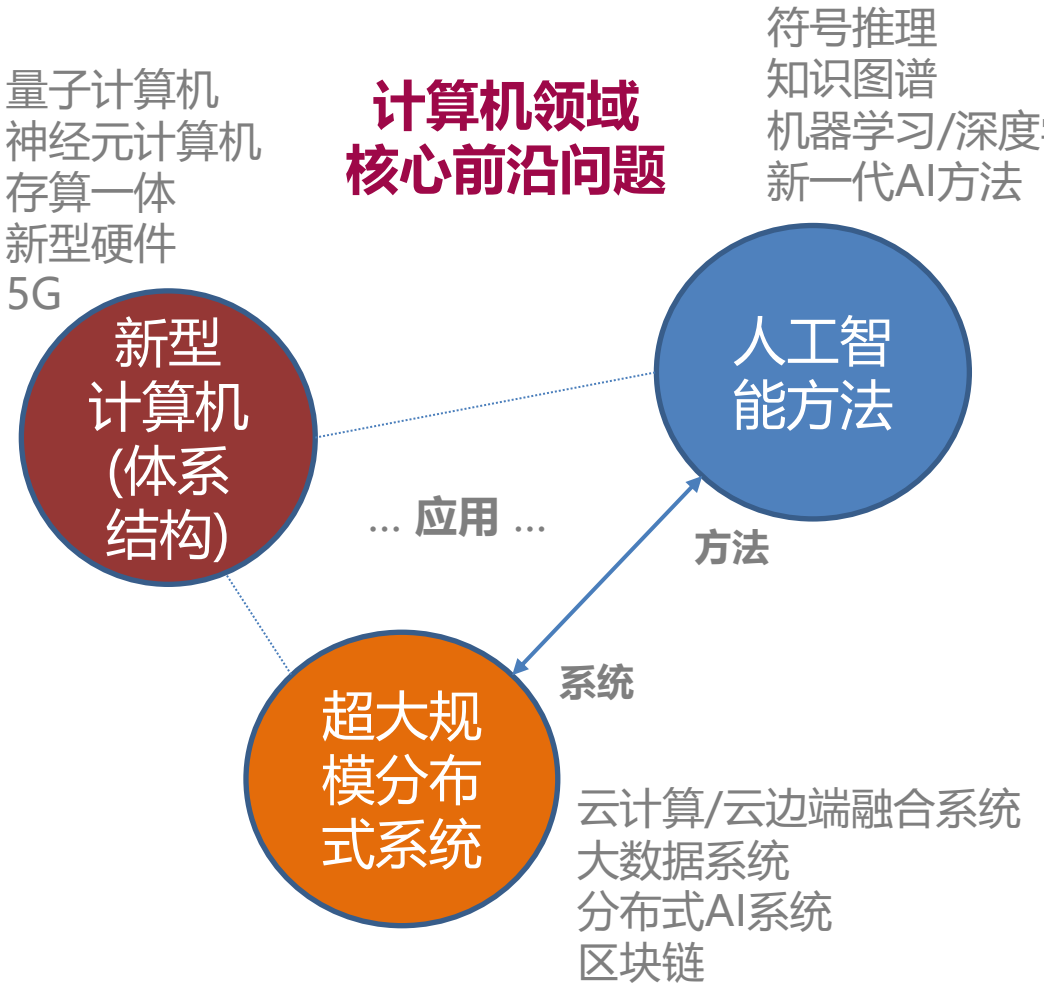
概念与特点

- **分布式系统的优缺点**
  - **优点**
    - 微处理机提供比大型主机更好的性能价格比 *(经济性)*
    - 匹配涉及空间分散机器的应用 *(固有分布性)*
    - 单个机器崩溃时系统还可以运转 *(可靠性)*
    - 计算能力可以逐渐有所增加 *(可扩展性)*
  - **缺点**
    - 目前为分布式系统开发的软件较少 *(软件复杂)*
    - 网络饱和或不稳定会引起系统故障 *(通信瓶颈)*
    - 容易造成对保密数据的访问 *(安全漏洞)*

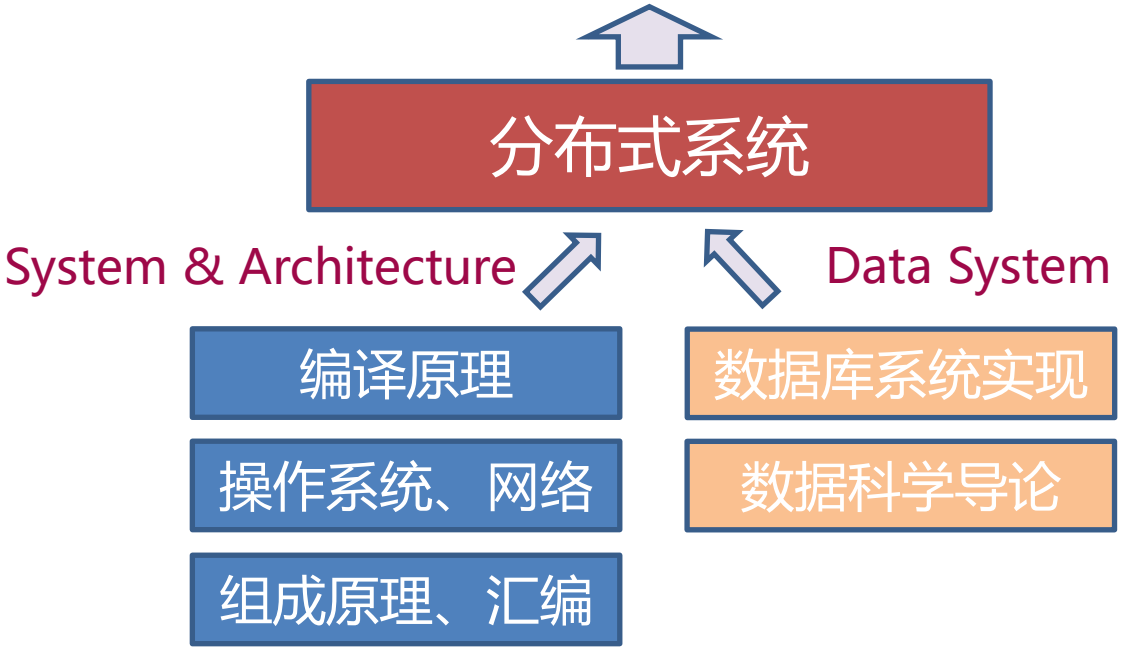


概念与特点

- **计算机专业知识体系**

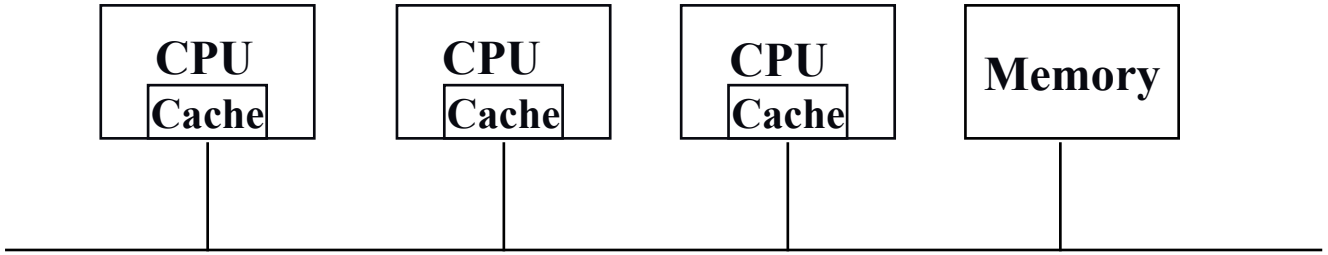


分布式文件系统，分布式数据库，分布式缓存，分布式计算系统，集群管理系统（Cloud OS），**分布式机器学习**，异构计算平台



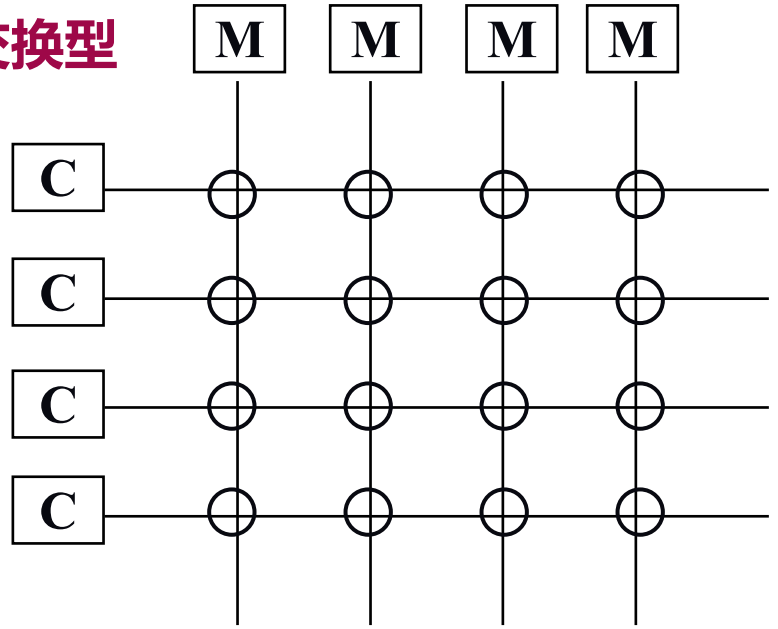
分布式系统发展史

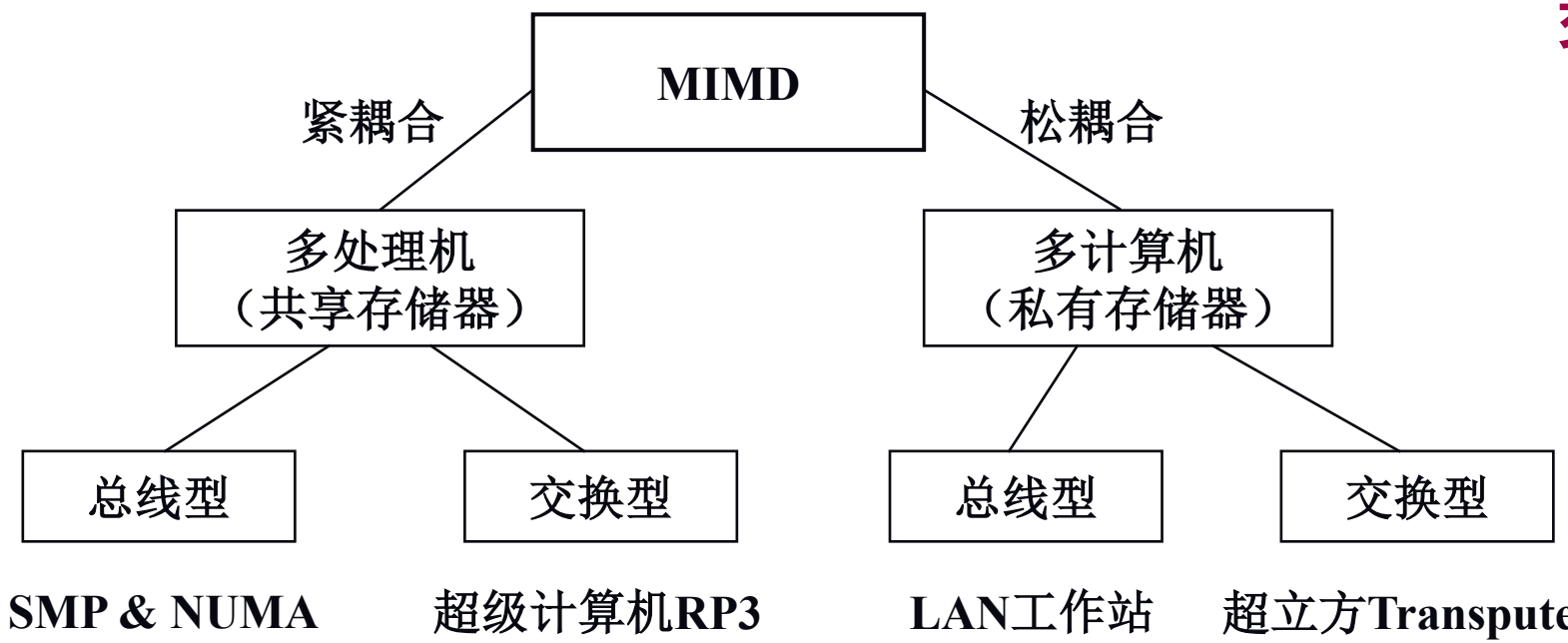
- **史前：并行/分布式计算机**
  - **计算机按指令集和数据流分类**
    - SISD（单处理机）-> SIMD（并行计算机）-> MIMD（分布式计算机）



总线型

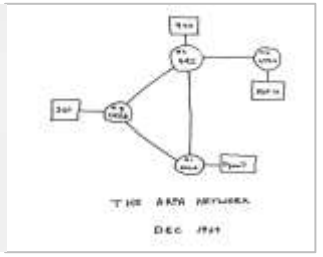
交换型





分布式系统发展史

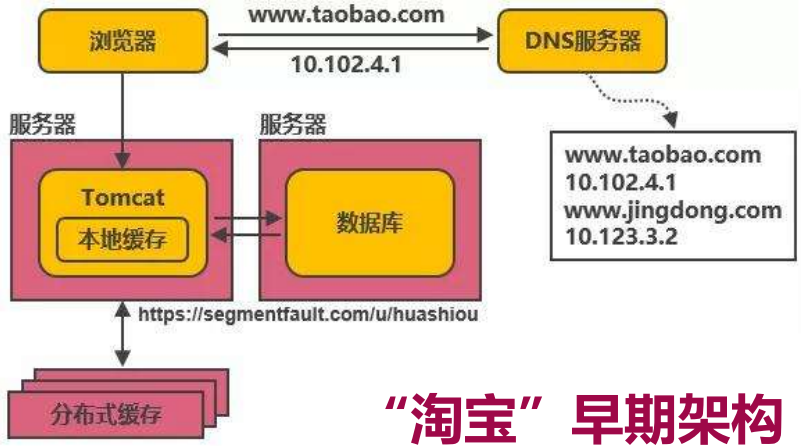




- **早期：分布式文件系统**
  - **起步**（1970s）
    - ARPANET（1968-1990，Internet前身，分组交换技术 & TCP/IP）
  - **第一代**（1980s）
    - 标准接口的远程文件访问，如NFS (SUN) 、AFS (CMU & IBM)
  - **第二代**（1990-1995）
    - 面向广域网设计，如SFS、Tiger Shark (IBM)
  - **数据库**（单机状态）
    - Oracle、IBM DB2、MS SQLServer、PostgreSQL、MySQL

分布式系统发展史

- **互联网时代开启：松散的“分布式”系统**
  - **第三代**（1990s后期）**& 第四代**（2000-）**分布式文件系统**
    - 第三代：Global File System (UMN)、GPFS (IBM)
    - 第四代：StorageTank (IBM)、Lustre (CFS)、BWFS (蓝鲸)、GFS (Google)
  - **广义分布式系统**
    - 互联网后台架构由多个系统拼接
    - 通过负载均衡、分库分表等方法，将负载分散到多机
    - 简单、实用，定制性比较强，但通用性不足
  - **数据库**（扩展模式）
    - 可以分库分表来扩展，如BigTable (Google)*（分布式需求还不强烈）*



“淘宝”早期架构

分布式系统发展史

- **当前：分布式系统大爆发**
  - **背景环境**
    - 移动互联网用户规模大爆发->底层分布式系统架构的强烈需求
    - 用户自研系统，基础软硬件公司（IBM & Oracle & EMC）缩水
    - 自研发降低成本、现有软件无法满足规模需求，按应用定制的收益巨大
  - **当前业界**
    - 先行者：Google（MapReduce、GFS、BigTable、Chubby、Borg）
    - 主力玩家1：大型互联网企业（Facebook、Twitter、阿里、百度、腾讯、头条、美团、滴滴、商汤等）
    - 主力玩家2：各大云平台（AWS、MS Azure、Google Cloud、阿里云、华为云、腾讯云、京东云、青云等）

分布式系统发展史

- **当前：分布式系统大爆发**
  - **分布式数据库**
    - NoSQL：Redis/Memcached、HBase、Cassandra、MangoDB（扩展性好，能满足用户增长，但牺牲了ACID）
    - 传统DB的分布式化：Greenplum（基于PostgreSQL）
    - NewSQL：Spanner -> CockroachDB、TiDB；Aurora、PolarDB
  - **其他分布式系统**
    - 分布式文件系统：HDFS、Lustre、Ceph
    - 分布式计算系统：Hadoop (MapReduce)、Spark、Flink
    - 分布式机器学习系统：TensorFlow
    - 云计算基础架构：Amazon系列；Borg->Kubernetes

挑战与目标

- **主要挑战**
  - **并发问题**（Concurrency）
    - 分布式计算系统*（无状态）*：任务分配均衡，数据交互/同步
    - 分布式存储系统*（有状态）*：数据分布决定，多副本有助于负载均衡
  - **局部故障**（Partial Failure）
    - 规模大出错是常态：网络数据包丢失、乱序、延迟，节点随时崩溃、失联
    - 要求更高：需要投入更多资源，甚至异地容灾 -> 更可靠 & 更可用
  - **性能问题**（Performance）
    - 性能是永恒的追求：更偏重扩展性而不是单节点能力，资源利用率和成本也要考虑
    - 系统更为复杂：会带来性能方面的副作用，如网络传输、多副本一致性、选举开销

挑战与目标

- **优化目标**
  - **性能**（Performance）
    - 可扩展性 (Scalability)：有合理的办法应对系统的增长（数据量、流量、复杂性）
    - 自设计/自调整能力：面临的用户访问工作量太复杂，智能选择模块/方法、参数调优
  - **故障容错**（Fault Tolerance）
    - 可靠性 (Reliability)：在硬件/软件故障、人为错误时仍能正确完成功能并达到期望性能
    - 可用性 (Availability)：在考察时间内系统能正常运行的概率或时间占有率期望
    - 可恢复性 (Resilience)：整个系统挂掉之后，再启动还能够恢复到正常的状态继续工作

挑战与目标

- **优化目标**
  - **一致性**（Consistency）
    - 分布式共识 *(空间)*：不同机器的多个副本一致（Paxos、Raft...）
    - 时间上一致性：先后到达请求获得的结果一致（线性一致性、因果一致性、最终一致性）
  - **服务质量**（QoS）
    - 相比传统的吞吐指标，更关注每个请求/租户的状态，一般用尾时延来衡量
    - 服务质量合约 (SLA)：指定服务所提供功能的期望状态的保障协定

挑战与目标

- **优化目标**
  - **效率/资源利用率**
    - 传统模式把资源尽可能多的转为性能，各类资源总有部分浪费，高峰预留利用率很低
    - 云计算模式利用虚拟化技术使底层硬件是可变的，弹性资源按需分配
  - **可维护性**（Maintainability）
    - 可操作性 (Operability)：便于运维团队操作
    - 简单性 (Simplicity)：便于读懂及继承修改代码
    - 可演化性 (Evolvability)：拥抱变化

本课目录

- 分布式系统概述
- **分布式通信**
- 分布式存储
- 分布式计算

分布式通信分类

- **基于服务模式**
  - **Web Service**：HTTP作为与服务通信的底层协议 (不限于Web，APP/中间件/OAuth)
  - **REST**（表述性状态转移）
    - 不是一个协议，而是一个基于HTTP原则的设计理念
    - 使用URL来标识资源，使用HTTP功能进行缓存控制、身份验证和内容类型协商
    - 强调简单的数据格式（符合该原则的API称为RESTful），经常与微服务相关
  - **SOAP**（简单对象访问协议）
    - 设计理念与REST完全相反：独立于HTTP，带有庞大而复杂的多种相关标准
    - 是一种基于XML的协议（该API使用称为WSDL，即Web服务描述语言）
    - WSDL的设计不是人类可读的，严重依赖于工具支持、代码生成和IDE

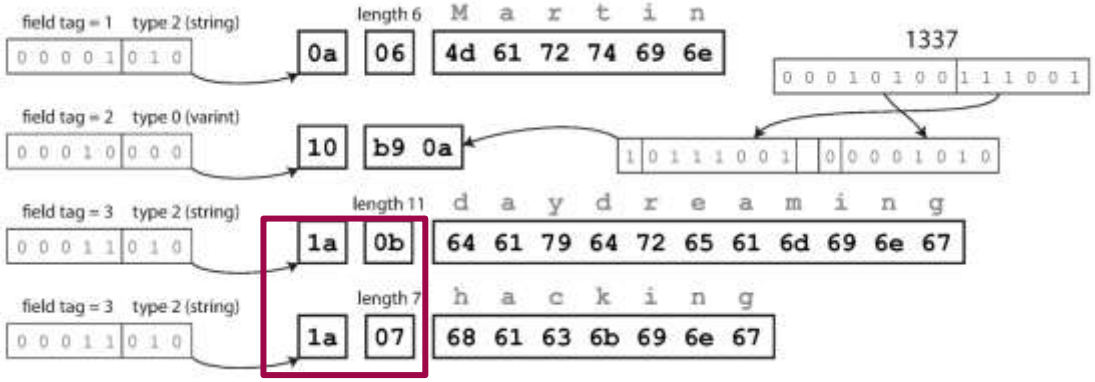
分布式通信分类

- **基于服务模式**

  - **RPC**（远程过程调用）
    - 使向远程网络服务发出请求看起来与本地调用函数相同（位置透明，1970s）
    - 比本地过程调用复杂：不可预测->网络慢 & 计算机故障、由于超时可能无结果
    - 在各种编码基础上构建RPC框架
      - gRPC是使用Protocol Buffers的RPC实现，支持流（一系列请求和响应）
      - Rest.li使用JSON over HTTP（RESTful API）
      - Finagle使用Thrift，Hadoop使用Avro

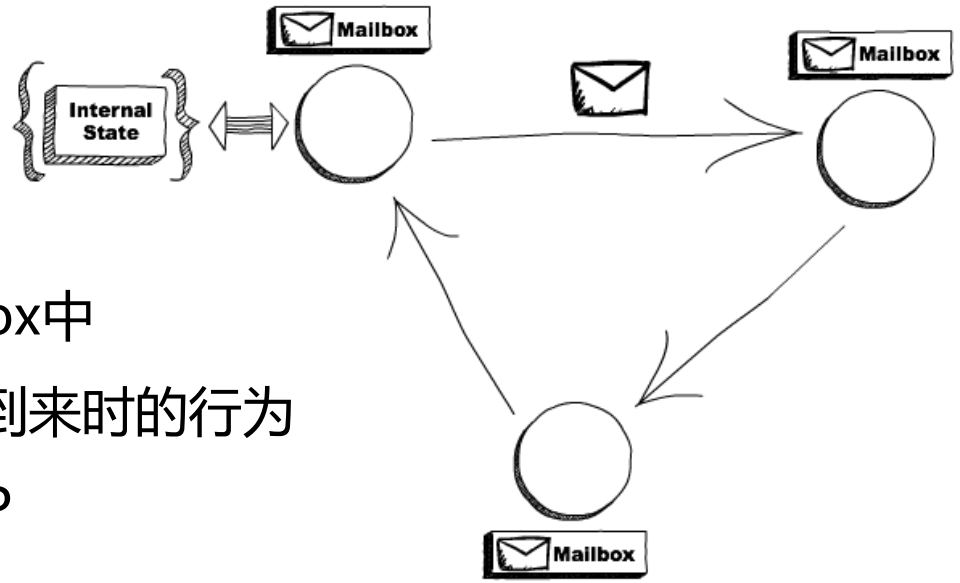
跨编程语言！

```
{
  "userName": "Martin",
  "favoriteNumber": 1337,
  "interests": ["daydreaming", "hacking"]
}
```



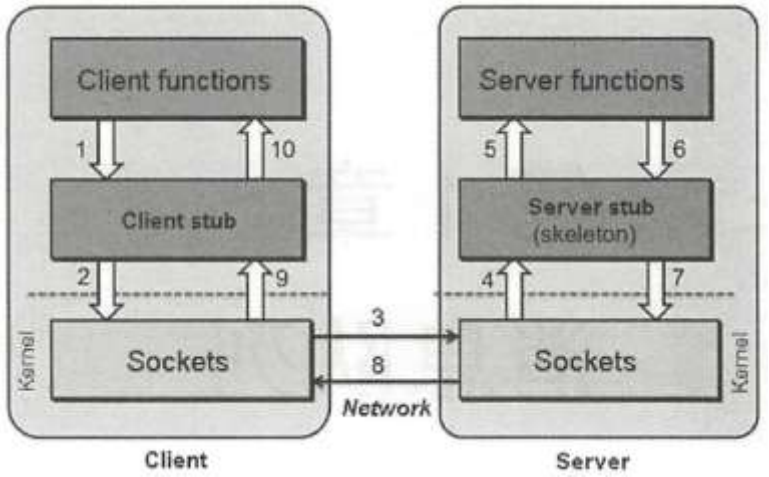
分布式通信分类

- **基于消息传递模式**
  - **消息代理**
    - 优点：消息缓冲，自动重发（可靠性）& 支持多播（避免互相知道太多信息）
    - 主要软件实现：RabbitMQ、ActiveMQ、Kafka、Redis
  - **分布式Actor框架**
    - Actor模型是一种并发模型
    - Actor接收消息进行计算，之间隔离不共享内存
    - 通过异步消息沟通，消息处理前被存放在Mailbox中
    - 三种状态：创建其他Actor、发消息、指定消息到来时的行为
    - 实际分布式框架：Akka、Orleans、Erlang OTP



分布式通信分类

- **基于消息传递模式**
  - **消息队列（MQ） vs RPC**
    - 两种通信模式：基于服务、基于消息传递
    - **RPC**：请求远程计算机上的服务（通信透明 & 跨语言）
    - **消息队列**：跨应用/节点的消息传递服务（隔离应用）
  - **消息队列作用**
    - 解耦（通知）、流量削峰（缓存并发请求）-> 异步处理
    - 日志收集系统（异步写入）、分布式事务（最终一致性）
  - **消息队列特点**
    - 消息持久化、可靠（重复）投递、严格有序、集群多播





消息中间件

- **概述**
  - **定义**
    - 一种独立的系统软件或服务程序，用于简化分布式应用间的数据传输，屏蔽底层的异构操作系统和网络平台（基于消息队列存储-转发机制）
  - **典型产品**
    - RabbitMQ (Mozilla)
    - ActiveMQ (Apache)
    - RocketMQ (阿里巴巴)

    开放协议
    - Kafka (Apache)
    - Redis (Merzia)

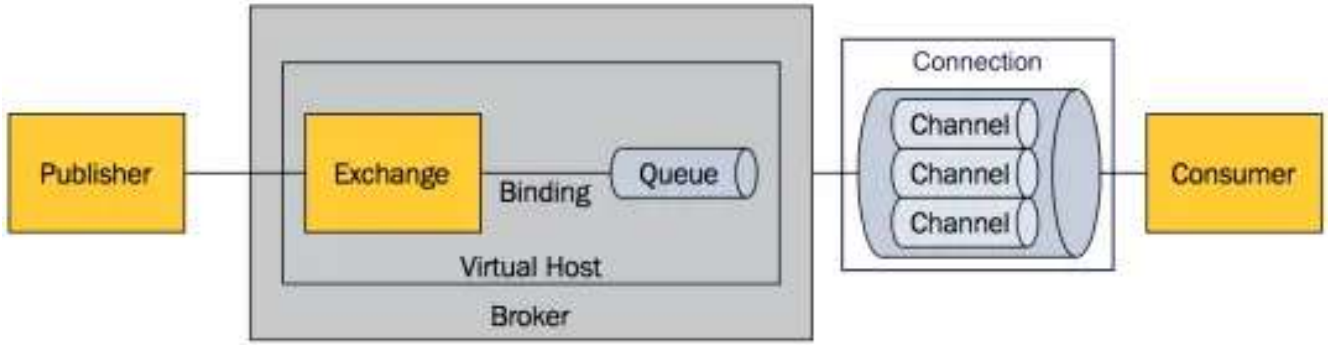
    私有协议

-	RabbitMQ	ActiveMQ	RocketMQ	Kafka
所属社区/公司	Mozilla Public License	Apache	Ali	Apache
成熟度	成熟	成熟	比较成熟	成熟
授权方式	开源	开源	开源	开源
开发语言	Erlang	Java	Java	Scala&Java
客户端支持语言	官方支持Erlang, Java, Ruby等, 社区产出多种语言API，几乎支持所有常用语言	Java、C、C++、Python、PHP、Perl、.net 等	Java C++（不成熟）	官方支持Java, 开源社区有多语言版本, 如PHP, Python, Go, C/C++, Ruby, NodeJS等编程语言, 详见Kafka 客户端列表
协议支持	多协议支持:AMQP，XMPP, SMTP, STOMP	OpenWire、STOMP、REST、XMPP、AMQP	自己定义的一套(社区提供JMS--不成熟)	自有协议，社区封装了HTTP协议支持
消息批量操作	不支持	支持	支持	支持
消息推拉模式	多协议, Pull/Push均有支持	多协议, Pull/Push均有支持	多协议, Pull/Push均有支持	Pull
HA	master/slave模式, master提供服务, slave仅作备份	基于ZooKeeper + LevelDB 的 Master-Slave 实现方式	支持多Master 模式、多 Master 多 Slave 模式，异步复制模式、多 Master 多 Slave 模式，同步双写	支持replica机制, leader宕掉后, 备份自动顶替，并重新选举leader(基于Zookeeper)
数据可靠性	可以保证数据不丢, 有slave用作备份	master/slave	支持异步实时刷盘，同步刷盘，同步复制，异步复制	数据可靠，并且有replica机制, 有容错容灾能力
单机吞吐量	其次(万级)	最差(万级)	最高(十万级)	次之(十万级)
消息延迟	微秒级	\	比kafka快	毫秒级
持久化能力	内存、文件，支持数据堆积,但数据堆积反过来影响生产速率	内存、文件、数据库	磁盘文件	磁盘文件, 只要磁盘容量够，可以做到无限消息堆积
是否有序	若想有序，只能使用一个Client	可以支持有序	有序	多Client保证有序
事务	不支持	支持	支持	不支持, 但可以通过Low Level API保证仅消费一次
集群	支持	支持	支持	支持
负载均衡	支持	支持	支持	支持
管理界面	较好	一般	命令行界面	官方只提供了命令行版, Yahoo开源自己的Kafka Web管理界面Kafka-Manager
部署方式	独立	独立	独立	独立

消息中间件

- **RabbitMQ**

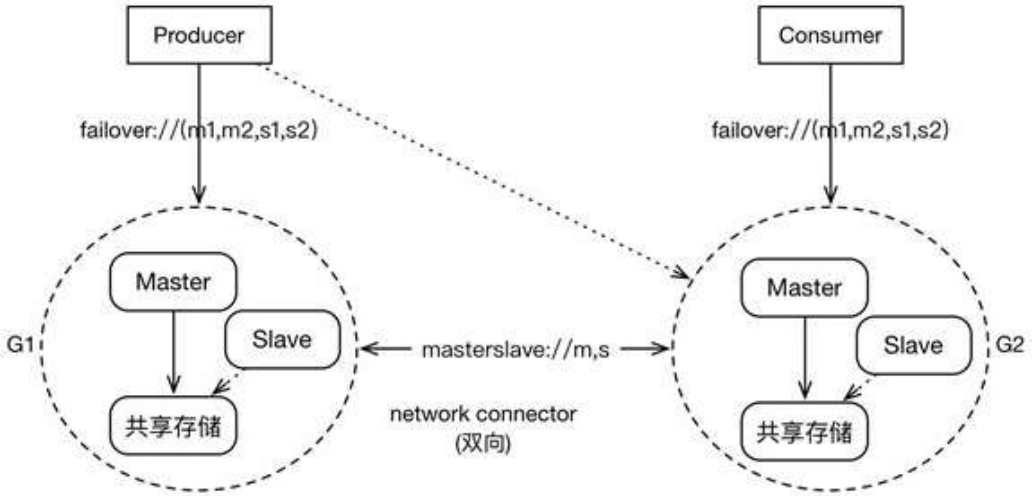
  - **简介**：基于Erlang开发的AMQP开源实现，Rabbit科技于2007发布（2010 VMware收购）
  - **特点**
    - **优点**：消息路由灵活，保证高可用性&可靠性，丰富插件、多协议/多语言客户端
    - **缺点**：吞吐量（消息堆积能力）较差 *（由于AMQP协议实现重量级）*
  - **消息路由**
    - 基于交换器和消息队列间的绑定
    - Direct：完全匹配路由键->单播
    - Fanout：不处理路由键->广播
    - Topic：通配符 (#, *) 匹配路由键



消息中间件

- **ActiveMQ**

  - **简介**：Apache发布的JMS v1.1开源实现（基于Java开发，支持J2EE v1.4规范）
  - **特点**
    - 提供高效、可扩展、稳定、安全的企业级消息通信
    - 支持多种语言客户端、协议和接口，附加消息组通信、优先级、延迟接收、持久化
  - **基本概念**
    - 消息模型：点对点、发布/订阅
    - 传输连接器：C2B，默认TCP + Failover
    - 网络连接器：B2B，静态配置 vs 动态广播
    - 消息存储：内存/文件/数据库 (默认KahaDB)



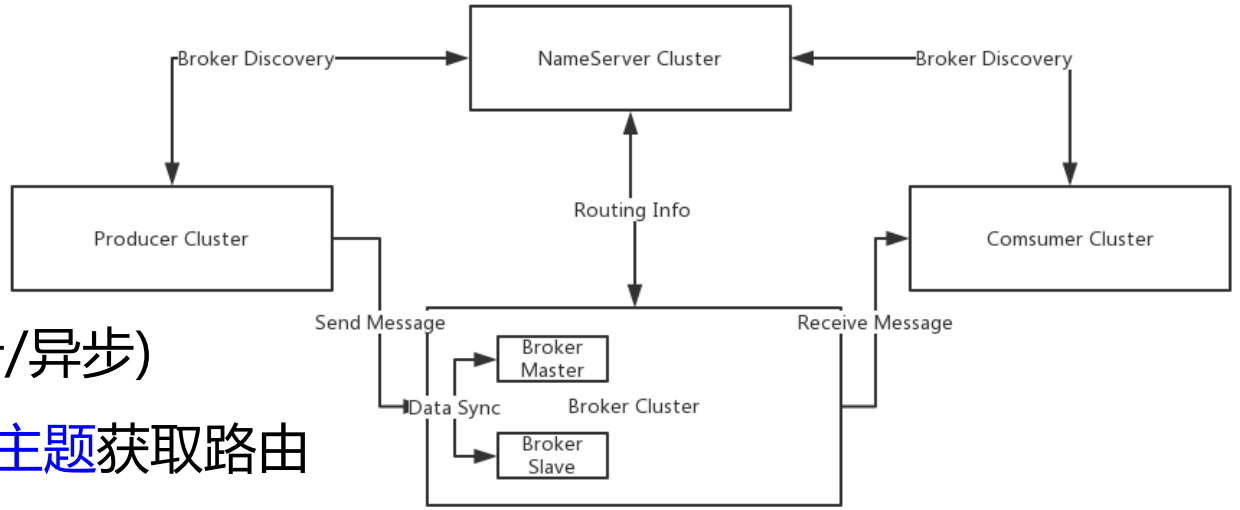
消息中间件

- **RocketMQ**

  - **简介**：阿里巴巴2012开发的国产中间件，2017成为Apache顶级项目

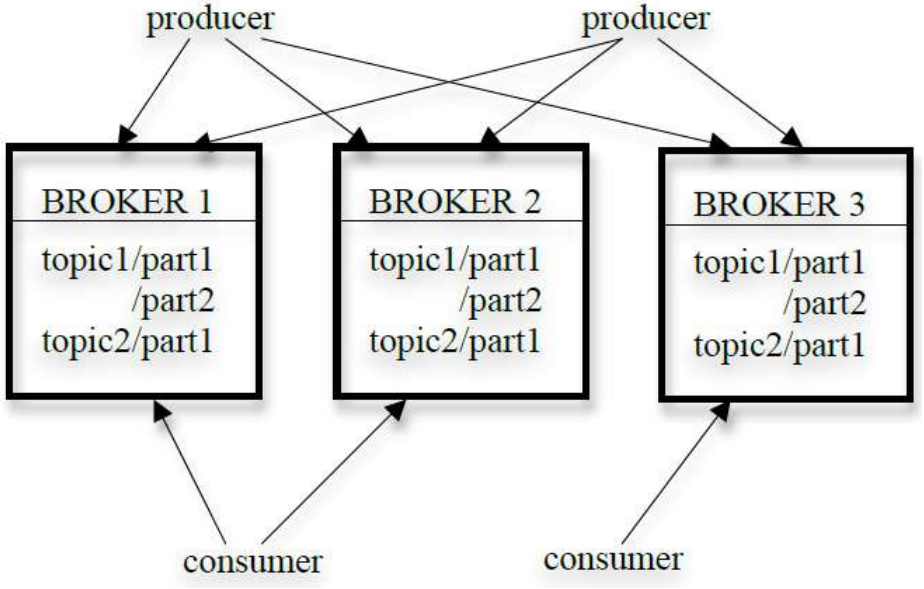
  - **特点**
    - 具有灵活的可扩展性 (集群)、支持顺序消息 (全局/局部有序) 和事务消息
    - 支持多种消息过滤 (代理服务器/消费者过滤)、支持回溯消费 (业务需要重新消费)

  - **架构组件**
    - 生产者(组)：同步/异步/单向发送
    - 消费者(组)：拉取型/推动型消费者
    - 代理服务器：单主/多主/多主从 (同步/异步)
    - 名称服务器：代理服务器注册->根据主题获取路由



消息中间件

- **Kafka**
  - **简介**：LinkedIn于2010用Scala开发的系统，2012成为Apache顶级项目
  - **作用**：解决海量高吞吐日志消息的处理（消息确认/原子写入不是必选项，会牺牲吞吐性能）
  - **架构组件**
    - 生产者：推模式发布消息 (访问或日志)
    - 消费者：拉模式订阅 & 消费消息 (应用服务)
    - 代理服务器：支持水平扩展提升吞吐率
    - ZooKeeper：管理代理服务器集群配置、选举领导者、消费者组变化时做分片均衡
    - 消息的类别称为主题 (Topic)，可分为多个分片 (Partition) 在不同代理服务器并行处理

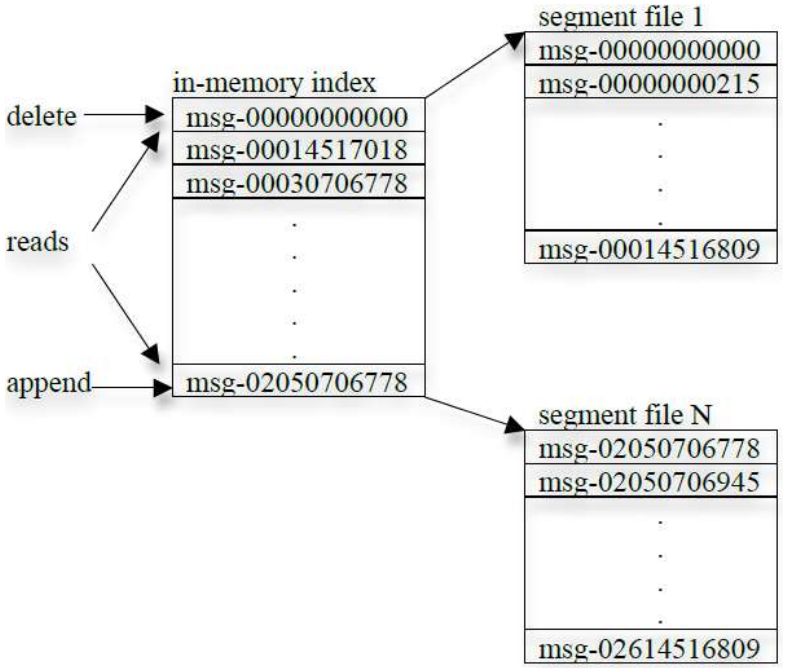


消息中间件

- **Kafka**

  - **分片设计**
    - 每个分片对应于一个文件夹，有多个段 (Segment)，各包含消息数据和索引文件
    - 分片数量由目标吞吐量推算，不小于消费者数量 (分片内消息对应同一消费者)

  - **消费偏移量**
    - 代理服务器不保存消息的状态，消费者轮询时返回由生产者写入但还未消费的消息
    - 消息在分片中顺序排列，通过偏移量确定位置 -> 分片的消费者变更时新消费者可以继续消费
    - 提交偏移量方式：自动 (默认)、手动同步/异步 -> 需避免消息被重复消费或遗漏消费

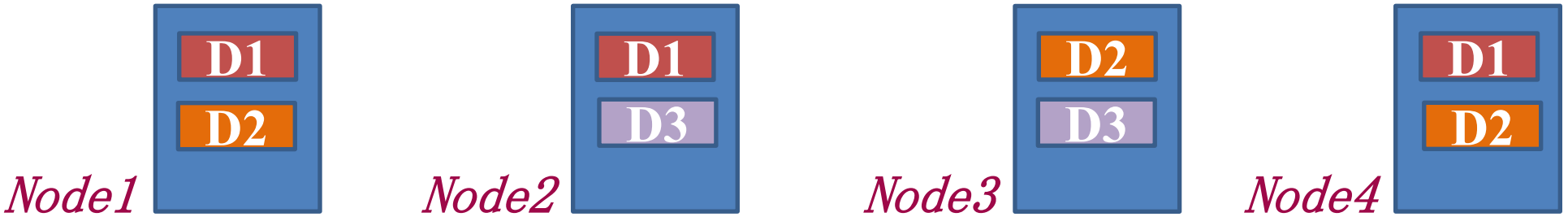


本课目录

- 分布式系统概述
- 分布式通信
- **分布式存储**
- 分布式计算

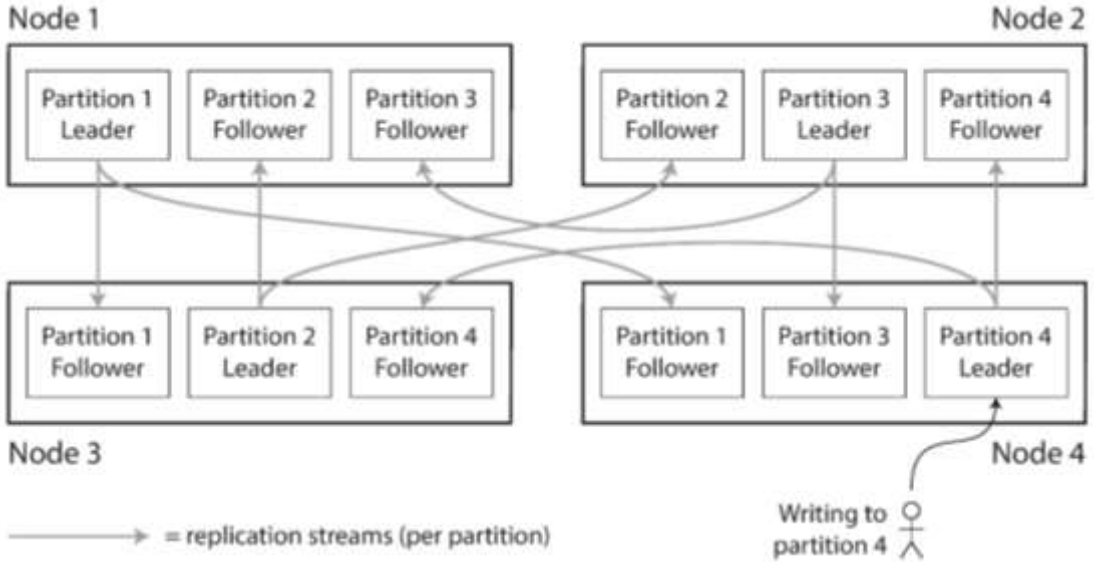
分布式存储

- **需要考虑的问题**
  - 用户如何找到某个数据块的正确位置？ *数据分片*
  - 如何让各个节点尽可能负载均衡来提升性能？
  - 多个副本之间如何保持一致？写成功（*Node1*）后，如何保障后续读（*Node4*）能够读到最新版本？ *数据复制*
  - 如何管理元数据？元数据服务器挂了呢？ *分布式共识*



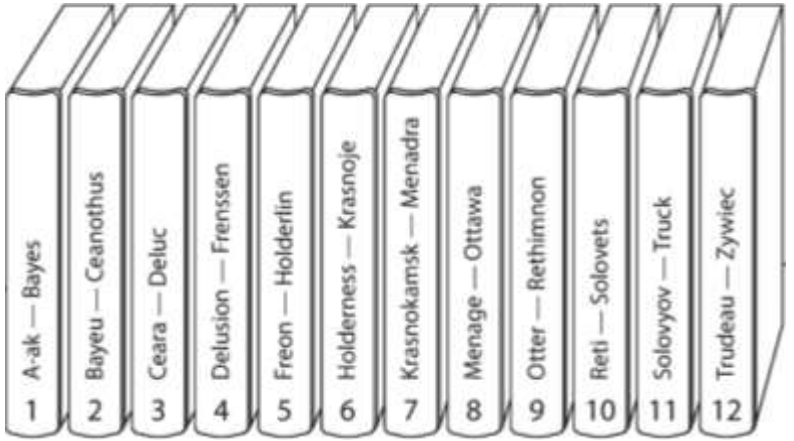
分布式存储

- **数据分片 vs 数据复制**
  - **区别**
    - **数据分片**：大数据集分散在不同节点上->提高可扩展性 & 增加查询吞吐量
    - **数据复制**：在不同节点保存相同数据的多个副本->服务高可用 & 防止数据丢失
  - **结合**
    - 主从复制模型与分片结合使用
    - 每个分片的副本存储在多个节点上



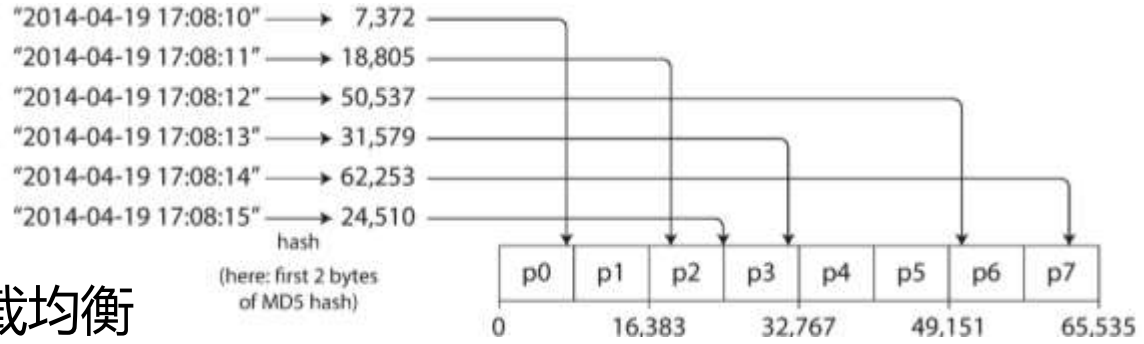
数据分片

- **键值数据的分片**
  - **分片要点**
    - **步骤**：对数据集进行逻辑划分 -> 将逻辑分片调度到物理节点
    - **要求**：尽量均匀，避免负载偏斜形成数据热点；分片后需要保留路由信息
  - **基于键的范围（Key Range）分片**
    - 每个分片指定一段连续的键范围 [最小值, 最大值]（类似百科全书按字母分卷）
    - **应用**：BigTable, HBase, RethinkDB, MongoDB
    - 一般搭配有序索引来存储（B+ tree, LSM-tree）
    - **优点**：可以进行快速范围查询（如一段时间）
    - **缺点**：某些访问模式（如时间戳）会导致热点



数据分片

- **键值数据的分片**
  - **基于键的散列（Hash）分片**
    - **优点**：随机分布，热点分散，更有助于负载均衡
    - **缺点**：丧失了高效执行范围查询的能力（有序相邻特性）
    - MongoDB：在所有分片上范围查询
    - Voldemort, Couchbase, Riak：不支持范围查询
    - Cassandra：由多列组成的复合主键, 第一列计算散列，后面有序可范围查询



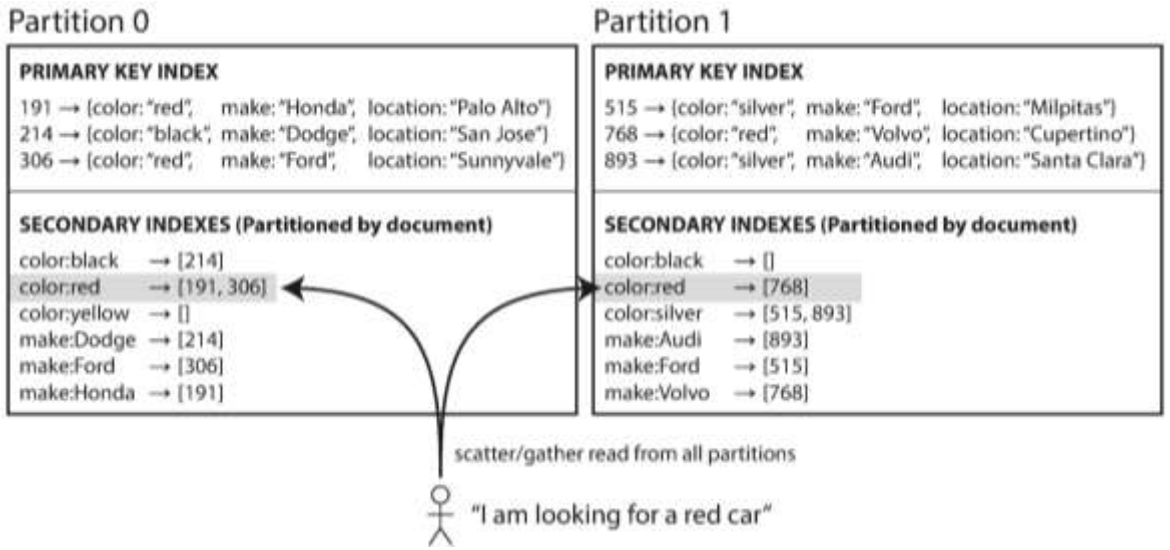
  - **负载偏斜和热点的消除**
    - 极端情况下所有读写操作都针对同一个键 -> 所有请求都被路由到同一个分片
    - 应用程序减少偏斜：在火爆的主键的开始或结尾添加一个随机数分配到不同分片

数据分片

- **语义分片**
  - **关系数据**
    - **按行划分**：便于更新，适合联机事务处理（OLTP）型应用
    - **按列划分**：便于读取、数据压缩，适合联机分析处理（OLAP）型应用
    - **行列混合划分**：在二者之间平衡（如Tile划分）
  - **图数据**
    - **点为中心**：与图操作更吻合，跨节点较少（如图导航、子图匹配找邻接节点）
    - **边为中心**：更有利于负载均衡（图中边数量一般远大于点的数量），点的度可能差距较大（如社交网络）
    - **切层**：属性较多的属性图

数据分片

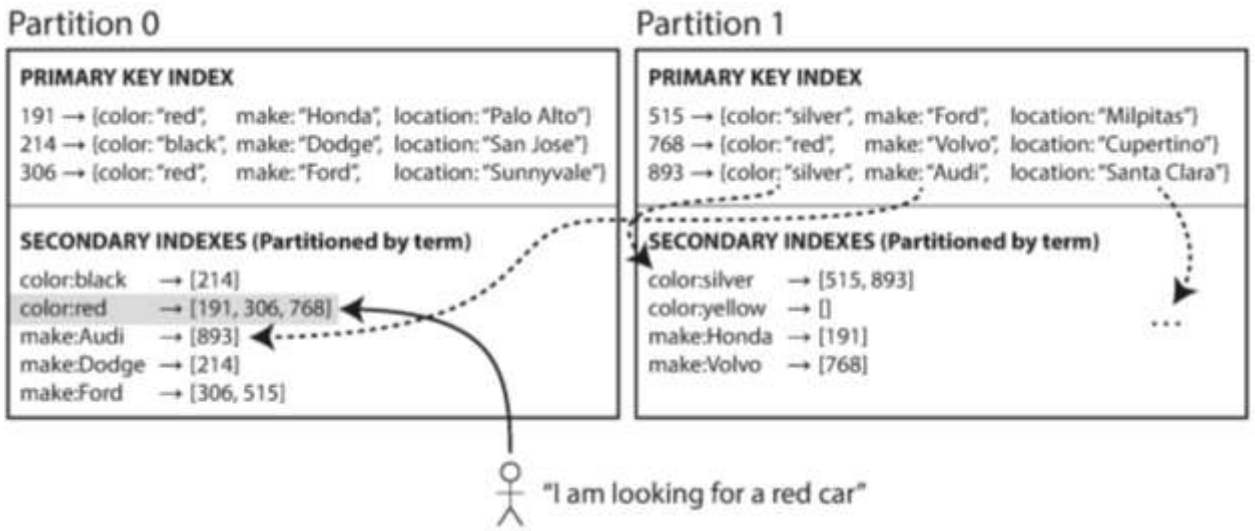
- **分片和次级索引**
  - **次级索引** (Secondary Index)：主键以外的列的索引，加速特定值查 (ID & Name)
    （HBase, Voldemort不支持，Riak支持）
  - **本地索引/基于文档** (Document) **索引**
    - 对每个数据分区独立地建立次级索引
    - 只针对本分片数据，不关心其他分片
    - **优点**：维护方便，分片内随数据更新
    - **缺点**：查询效率低-> 分散/聚集
      - 多分片并发查询面临长尾请求
    - **应用**：MongoDB, Riak, Cassandra等



数据分片

- **分片和次级索引**
  - **全局索引/基于关键词 (Term) 索引**
    - 构建覆盖所有分片数据的索引->为了避免查询热点也分片存储，可与主键不同
    - **优点**：读取更有效率，不需要分散/收集
    - **缺点**：写入速度慢且复杂，写入影响索引的多个分片
    - 异步更新降低延迟 -> 数据不一致
    - 跨分片的分布式事务：性能损失

Trade-off

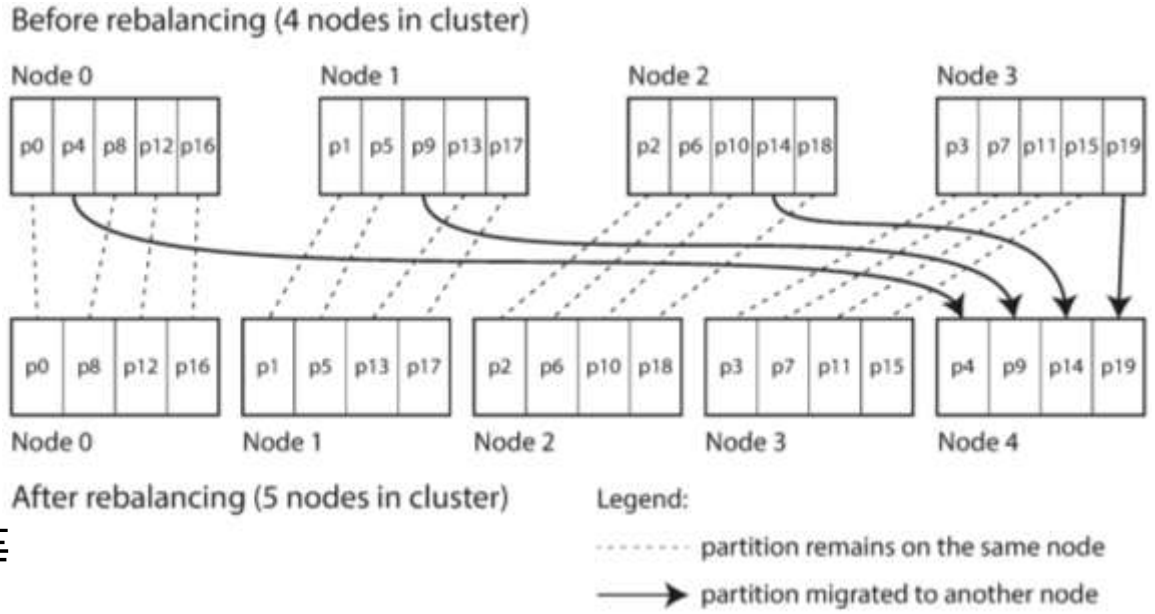


数据分片

- **分片均衡**（Rebalancing）
  - **数据分片考虑因素**
    - 分片之间的相关性（如图查询）-> 相邻的分片尽可能在同一节点上
    - 同一分片的不同副本的分布，需要考虑可靠性/可用性 -> 异地容灾
    - 负载均衡以提升性能/扩展性 -> 动态增减节点 & 数据迁移（开销较大）
  - **均衡的要点**
    - **触发条件**：查询压力增加需要更多机器处理、数据规模增加需要更多磁盘和内存存储、节点故障需要其他机器接管
    - **实现目标**：均衡后数据存储和读写请求在集群中更均匀分布，均衡过程中可以正常提供读写服务，避免不必要的数据迁移以加快均衡并尽量降低网络和I/O影响

数据分片

- **分片均衡**（Rebalancing）
  - **反面教材：对节点数取模**（mod）
    - 将散列值划分为不同的区间范围，每个区间一个分片 *(Hash(key) mod N)*
    - 节点数增减会导致很多数据迁移到其他节点 -> 频繁迁移增加均衡成本
  - **固定数量的分片**
    - 创建远超节点数的分片数
    - 每个节点分配不定个数的分片
    - 节点数变化时只需迁移少量分片
    - 关键字到分片的映射关系不变
    - 应用：Riak, Couchbase, Voldemort等

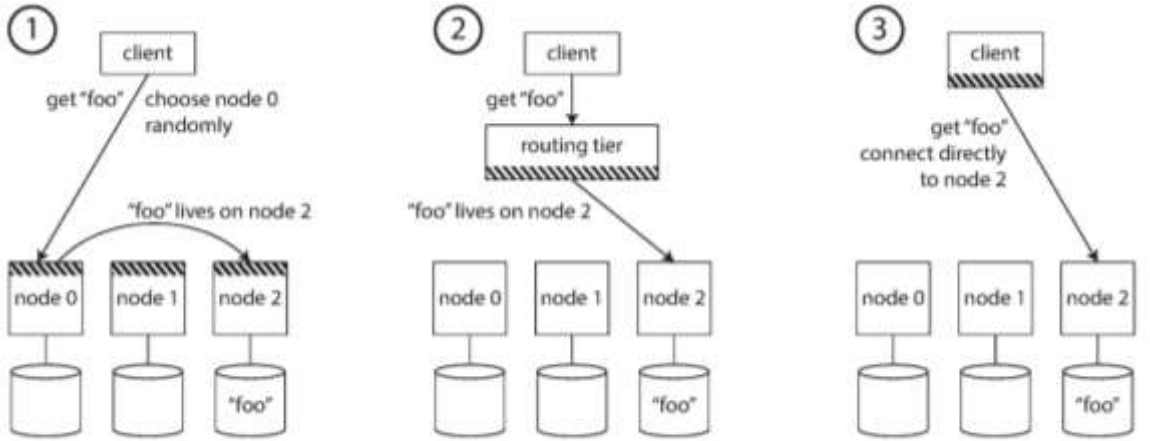


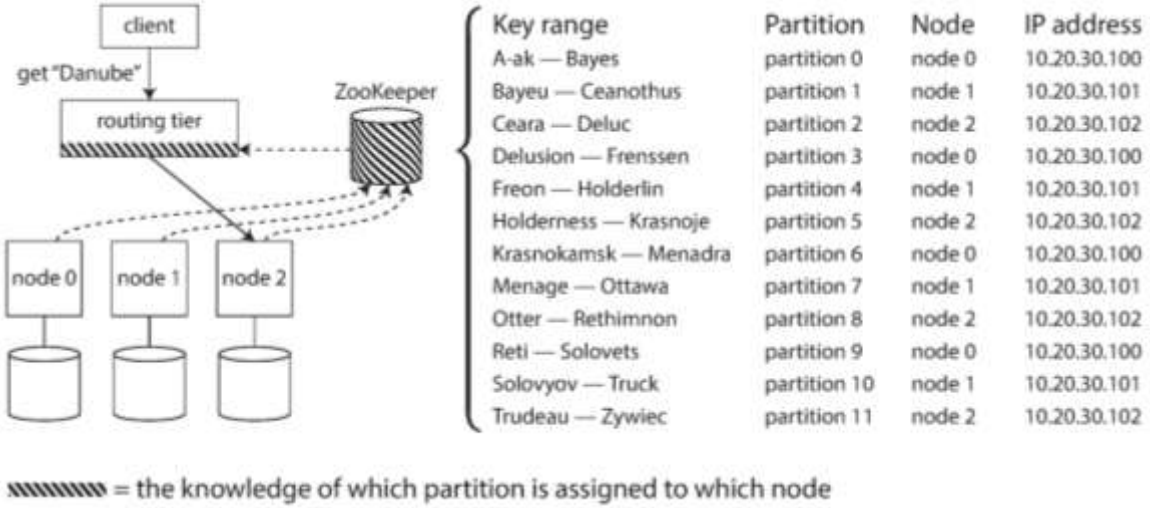
数据分片

- **分片均衡**（Rebalancing）
  - **动态分片**
    - 每个分片负责一个键范围内的数据，如果数据过多则分片拆分为两个 -> 分片数量自动适应数据总量（每个节点维护不定个数的分片，拆分的分片可转移节点）
    - 应用：HBase（分裂阈值10GB），MongoDB, RethinkDB等
  - **按节点比例分片**
    - 分片数与集群节点数成正比，即每个节点维护的分片数为常数
    - 增加节点数时分片变小 -> 新节点随机选择固定数量的现有分片拆分，占有这些每个分片的一半，另一半留在原节点
    - 应用：Cassandra（默认每个节点有256个分片），Ketama
  - 半自动运维均衡：系统给出迁移策略，管理员审核决定何时运行 (Riak, Voldemort)

数据分片

- **请求路由**
  - **每个节点有全局路由表**
    - 一致性哈希算法：Dynamo
    - Gossip协议同步状态：Cassandra
  - **有一个专门的路由层**
    - 主从架构 (GFS)
    - 路由层不处理请求
    - 协调服务：HBase, MongoDB
  - **客户端感知分片和节点映射**
    - 分布式缓存 (Memcached)

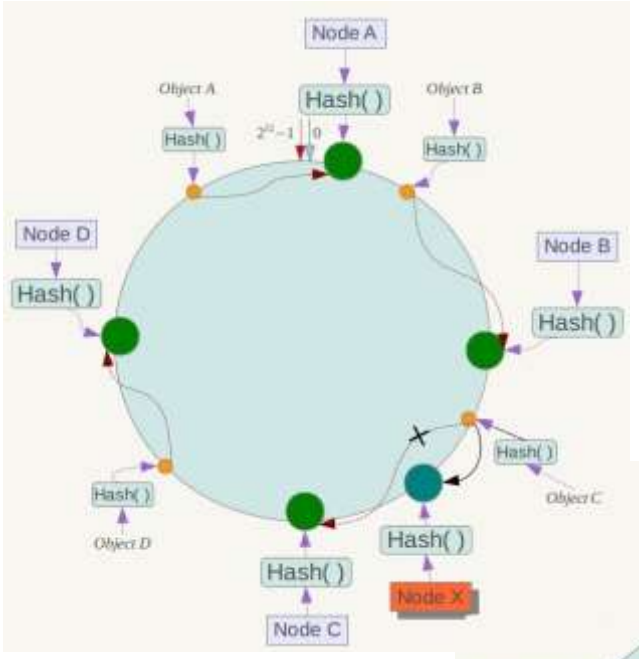




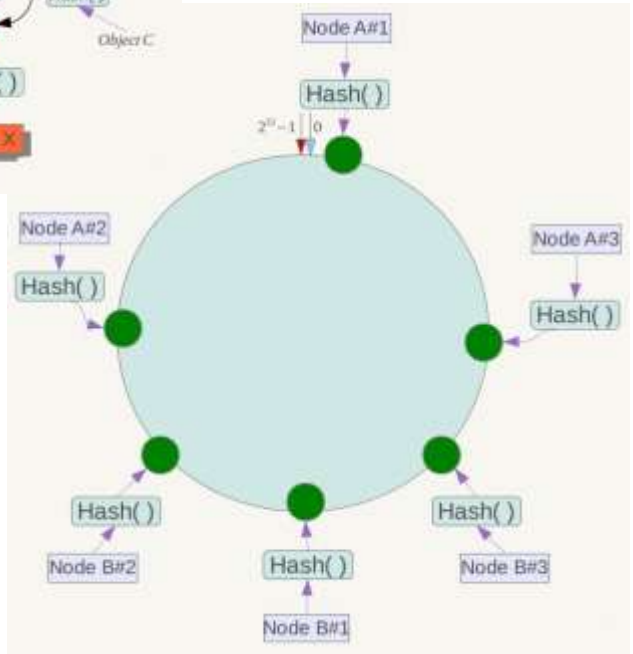
Key range	Partition	Node	IP address
A-ak — Bayes	partition 0	node 0	10.20.30.100
Bayeu — Ceanothus	partition 1	node 1	10.20.30.101
Ceara — Deluc	partition 2	node 2	10.20.30.102
Delusion — Frenssen	partition 3	node 0	10.20.30.100
Freon — Holderlin	partition 4	node 1	10.20.30.101
Holderness — Krasnoje	partition 5	node 2	10.20.30.102
Krasnokamsk — Menadra	partition 6	node 0	10.20.30.100
Menage — Ottawa	partition 7	node 1	10.20.30.101
Otter — Rethimnon	partition 8	node 2	10.20.30.102
Reti — Solovets	partition 9	node 0	10.20.30.100
Solovyov — Truck	partition 10	node 1	10.20.30.101
Trudeau — Zywiec	partition 11	node 2	10.20.30.102

数据分片

- **一致性哈希算法**
  - **基本设计**
    - 环形哈希空间：$0 \sim 2^{32}-1$
    - 服务器/数据映射：计算哈希函数在环上定位 -> 数据存储在顺时针最近的服务器
    - 服务器增减 -> 重新分配数据到顺时针最近的服务器

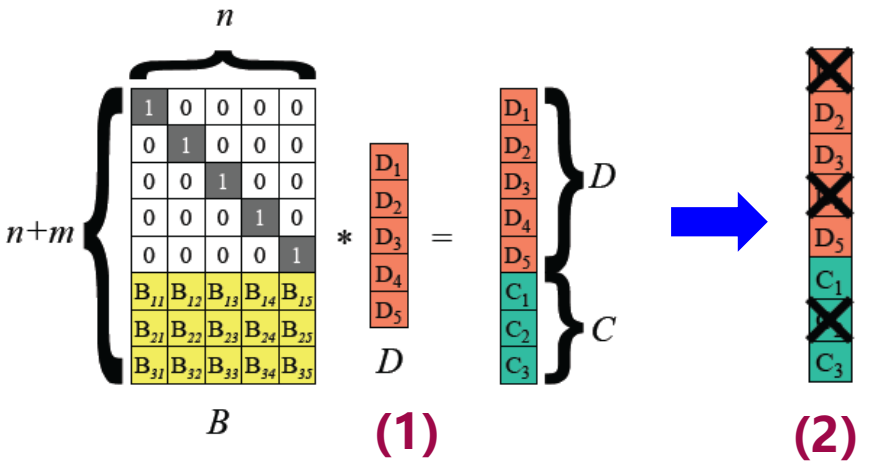


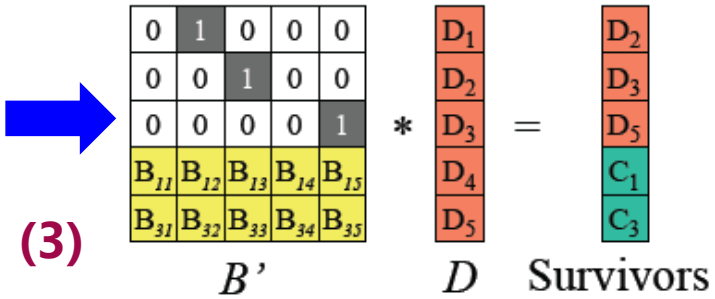
  - **均衡问题**
    - 服务器映射在环的一侧 -> 数据可能都存储在一台服务器
    - 方案：确定服务器物理节点关联的虚拟节点，增加编号后计算哈希在环上定位（更均匀）-> 数据/节点二级映射

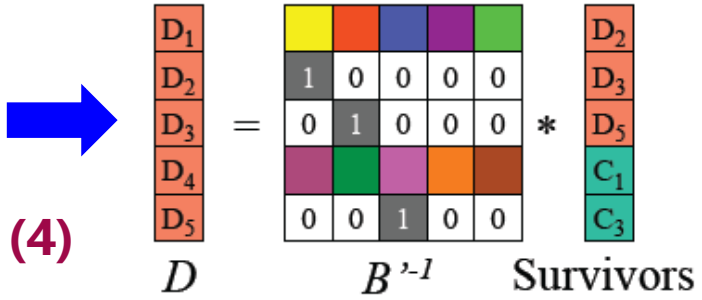


数据复制

- **复制分类**
  - **多副本**（Multi-Replica）
    - 直接复制到多机，适合读写数据，存储量较大
    - 分类：主从复制、多主节点复制、无主节点复制
  - **纠删码**（Erasure Coding）
    - 将n份数据增加m份数据，通过任意n份数据还原原始数据
    - 编码以计算换存储，适合只读数据，存储量较小
    - 分类：阵列纠删码（RAID5）、RS纠删码、LDPC纠删码

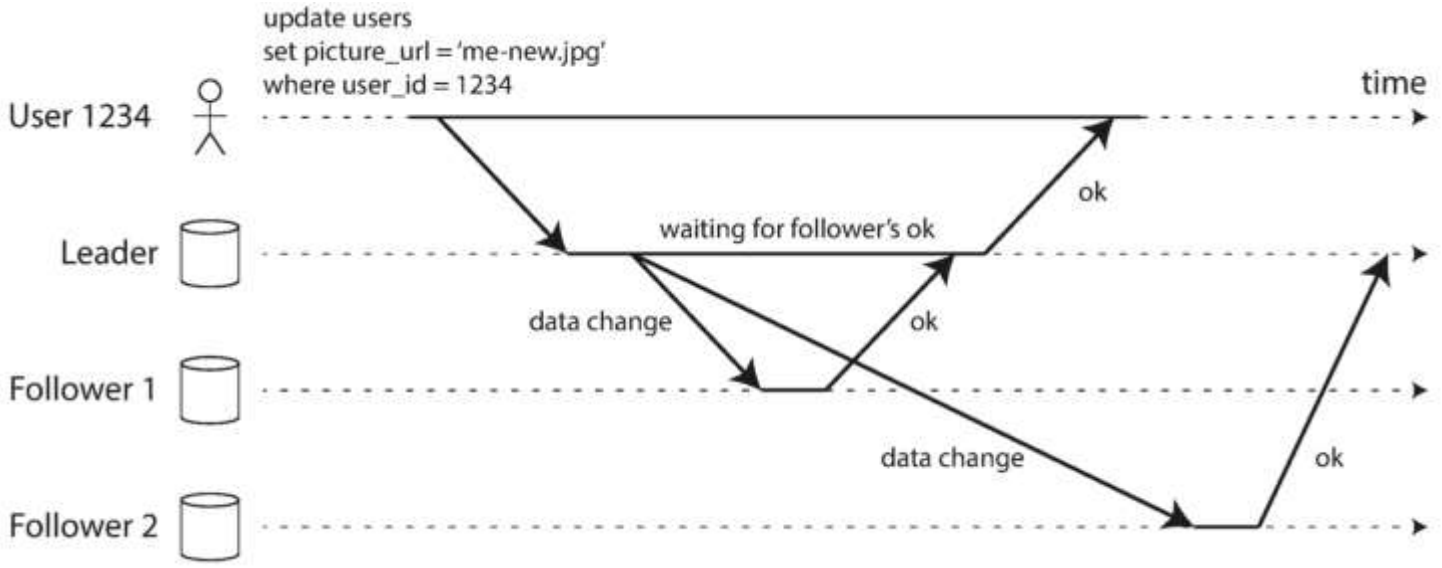






数据复制

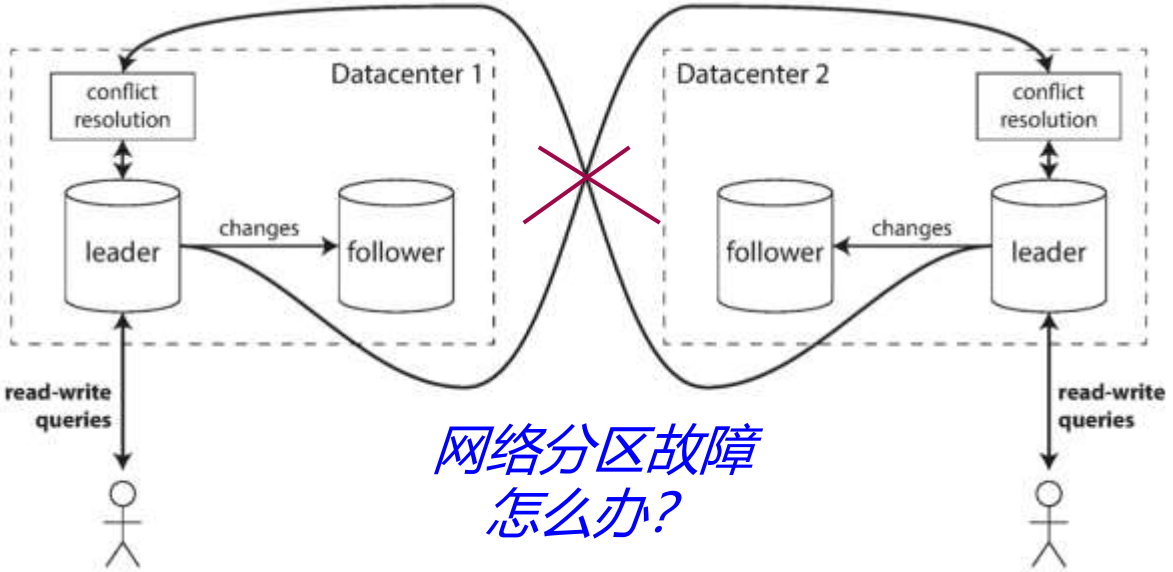
- **典型复制方法**
  - **主从复制**
    - 一主多从，从节点可以扩展读性能 -> 主副本/从副本
    - 同步复制 vs 异步复制：区别在于请求何时返回客户端，是否等待副本写完?
    - 从节点新增：快照+日志复制
    - 主节点失效：选举+切换



数据复制

- **典型复制方法**
  - **多主节点复制**
    - 多个可以接受写入的主节点，每个主节点写入后都转给其他所有节点
    - 拓扑结构：环型、星型、All-to-All拓扑
    - 冲突写处理：合并、高优先级、后者胜、保留冲突

对比项	单主模型	多主模型
性能	所有写入都要路由到一个DC	写入可以就近
可用性	主副本所在DC故障，需要切换	每个数据中心可以独立工作
网络	跨DC写入对网络抖动更敏感	DC间异步复制容错性更高

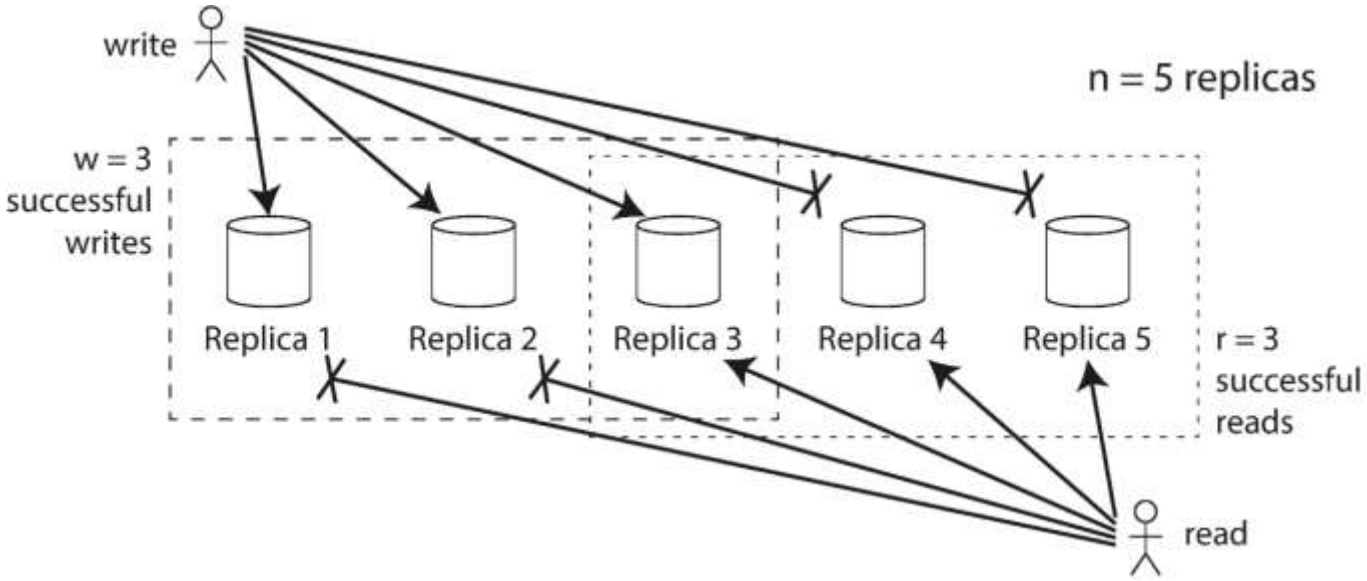


网络分区故障怎么办?

数据复制

- **典型复制方法**
  - **无主节点复制**
    - 去中心复制，客户端直接发写请求到多副本，读取多个副本选取最新版本值
    - Quorum机制：n个副本，写入时需要w个节点确认，读取时至少查询r个节点
      -> 只要w+r>n，则可保证总能读到最新数据

成功写入的节点集合和读取的节点集合必然有重合

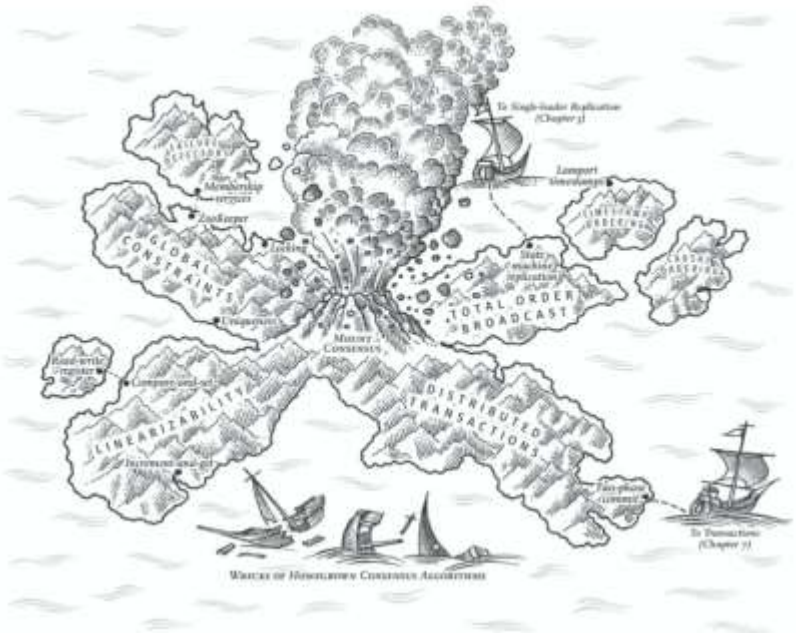


分布式存储

- **分布式一致性**
  - **线性一致性**（Linearizability）
    - 提供只有一个副本的假象，并且所有的操作都是原子性的
    - 只要成功完成写操作，所有从系统中读取数据必须能够看到刚刚写入的值
    - 最高等级一致性，也称原子一致性、强一致性、立即一致性、外部一致性
  - **因果一致性**（Casual Consistency）
    - 建立因果顺序使得系统可以在消息无序传递的情况下重新构建事件序列
    - 填补消息之间的空隙，避免在某些消息仍然缺失的情况下发布操作结果
  - **最终一致性**（Eventual Consistency）
    - 系统中所有的数据副本在经过一段时间的同步后最终达成一致的状态

分布式存储

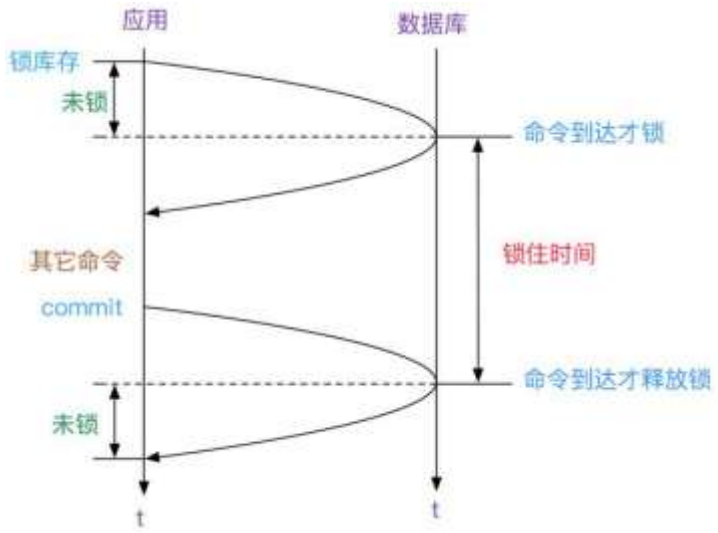
- **分布式共识**
  - **概念**
    - 广义：让所有的节点对某件事达成一致 vs 狭义：多个副本之间达成一致
    - 挑战：需要全面考虑所有出错情况，如脑裂（两个节点都认为自己是领导者）
  - **Paxos家族**：唯一严格证明的分布式共识算法/协议
    - 基本Paxos算法
    - Multi-Paxos算法（Chubby）
    - Raft算法
    - ZAB协议（ZooKeeper）



- **分布式共识**
  - **Paxos**
    - 聚焦于解决确定一个值的单一决策问题，需通过组合多个实例实现日志复制
    - 理论严谨但实现复杂，需开发者自行处理状态管理，灵活性高但易引发冲突
  - **Raft**
    - 面向日志复制，将问题分解为Leader选举、日志复制和安全性三个独立模块
    - 通过强制角色分离和明确流程（心跳机制）降低实现难度，减少冲突概率
  - **ZAB**
    - 能保证操作顺序性、面向ZooKeeper基于主从模式的原子广播协议
    - 侧重高吞吐量事务，用于分布式锁等高频读场景，读操作默认最终一致性

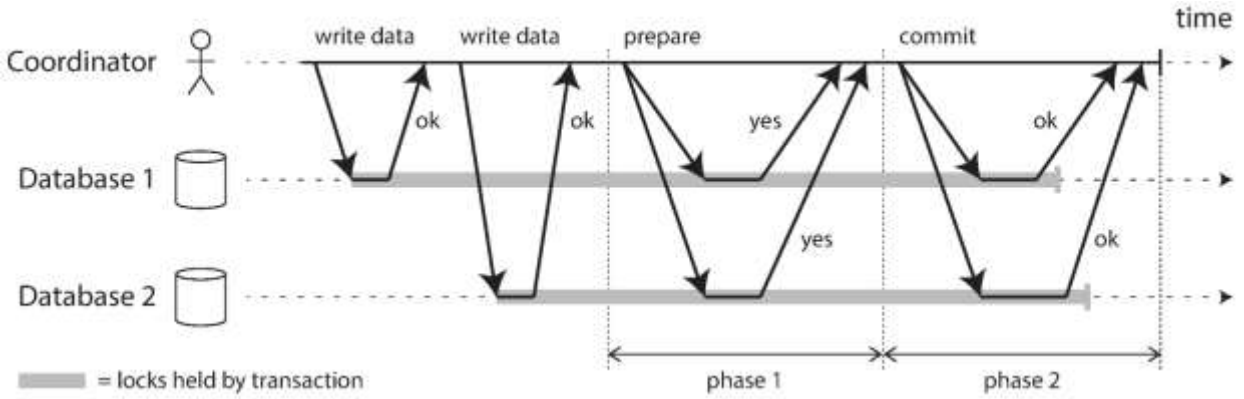
分布式存储

- **分布式事务**
  - **要点**
    - 安全性保证：ACID（原子性、一致性、隔离性、持久性）
    - 修改多个对象，使用事务ID对操作分组



  - **事务隔离**
    - 读已提交：保证读/写数据库时，不能看到/覆盖未成功提交的数据 (脏读/脏写) -> 提交前返回旧版本值+写锁 *（读倾斜）*
    - 快照隔离：采用多版本并发控制 (MVCC) 保留不同版本，读事务只能访问所有版本<=txid的数据 *（写倾斜）*
    - 可串行化：保证任意时刻只有一个事务在运行
    - 两阶段加锁（2PL）：事务执行前为获取锁阶段，提交/中止后为释放锁阶段

- **分布式事务**
  - **两阶段提交（2PC）**
    - 多节点间阻塞式原子提交协议，确保所有节点要么全部提交，要么全部中止
    - 协调者在准备阶段发送准备请求到所有参与者询问是否可以提交并跟踪回复
      - 所有参与者回复“是” -> 协调者在提交阶段发出提交请求，执行提交
      - 有任何参与者回复“否” -> 协调者在提交阶段向所有参与者发出中止请求
  - **三阶段提交（3PC）**
    - 两阶段提交的准备阶段细分为两个阶段 ->减少阻塞时间
    - 引入超时，参与者等待超时默认提交->解决协调者故障阻塞

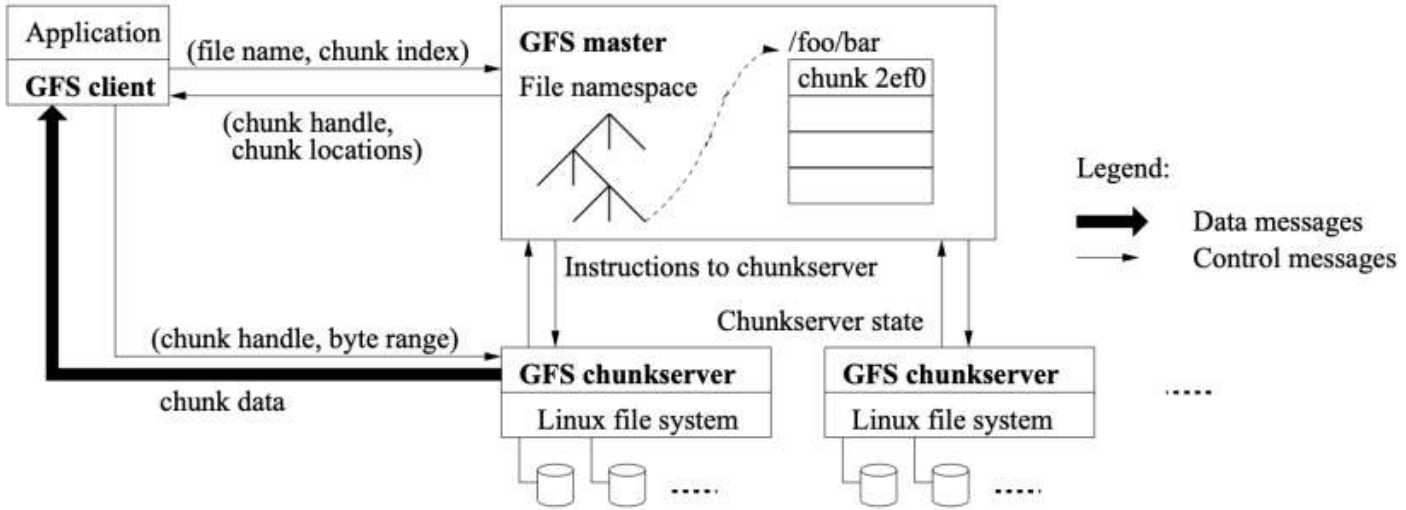


分布式文件系统

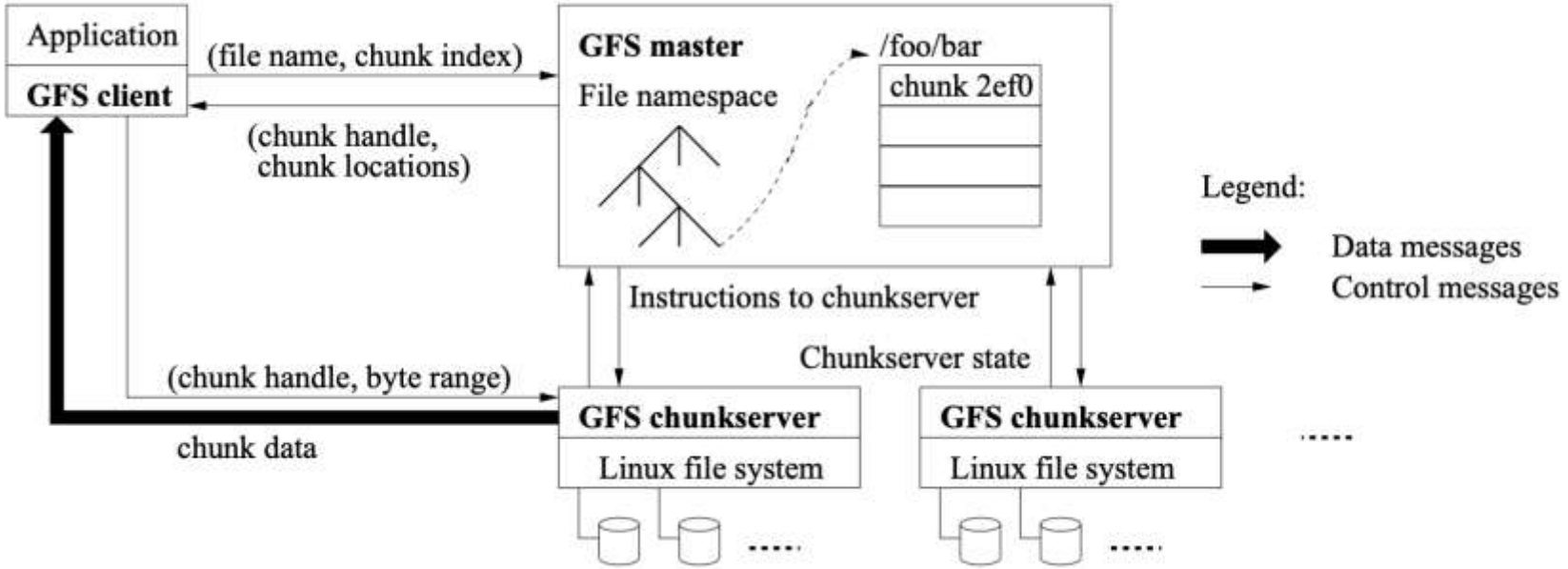
- **Google文件系统（GFS）**
  - 基于大量廉价经常发生故障的商用组件构建（多副本容错）
  - 面向GB数量级大文件管理（文件块大小设为64MB）
  - 包括两种类型读操作：大规模(MB)数据流读、小规模(KB)随机读（尽可能合并）
  - 写操作通常是追加（Append）操作（优化性能且保证原子性）
  - 设计机制保证许多客户向同一个文件进行并发追加操作的正确性
  - 高稳定的大数据量操作（带宽）比低延迟更重要

针对Google公司实际应用中遇到的问题提出

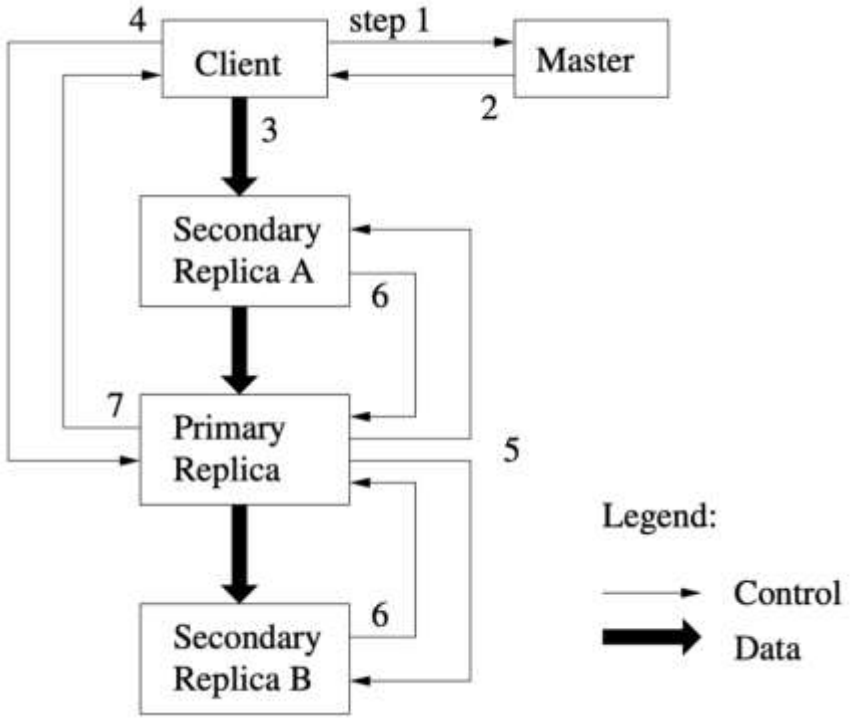
- **Google文件系统（GFS）**
  - **系统架构**
    - 多个客户端（Client）：不支持POSIX接口，而是单独的访问API（create, delete, open, close, read, write, *snapshot*, *record append*）
    - 单个主控服务器（Master）：文件名 -> 块地址（块服务器），不处理数据流
    - 多个块服务器（Chunkserver）：响应块查询返回块数据，多副本放置



- **Google文件系统（GFS）**
  - **读流程**
    - 客户端先联系主控服务器定位到块服务器（客户端会有块地址缓存，但有效期有限）
    - 客户端直接访问块服务器获取数据（不一定是主副本，任意一个均可，一般是网络距离更近的）



- **Google文件系统（GFS）**
  - **写流程**
    - 客户端需要同步更新所有副本才能完成写操作
    - (1~2) 客户端从主控服务器获取块副本位置和租约
    - (3) 客户端将写入数据发到相关块服务器缓存
    - (4) 客户端向主副本所在块服务器发写请求
    - (5~6) 主块服务器排定修改顺序，控制写入辅助副本
    - (7) 主块服务器将信息反馈客户端，出错则重复(3~7)
    - 每次写完成后块版本号+1 -> 用于检测过时的块副本



分布式文件系统

- **Google文件系统（GFS）**
  - **控制设计**
    - 元数据：文件和块的命名空间、文件到块的映射信息、块副本的位置信息（不多于64B，块大小64MB）
    - 操作日志：唯一持久化，全局有序以解决并发冲突，基于检查点机制　　*/d1/d2/.../dn/leaf*
    - 命名空间：管理完整路径名到元数据的查找表，通过前缀压缩放在内存中
    - 心跳消息：主控服务器与各块服务器周期性通信 -> 给出指令 & 收集状态
    - 垃圾回收：定期扫描元数据，延迟回收过时数据 *(版本号检测停机导致的过期副本)*

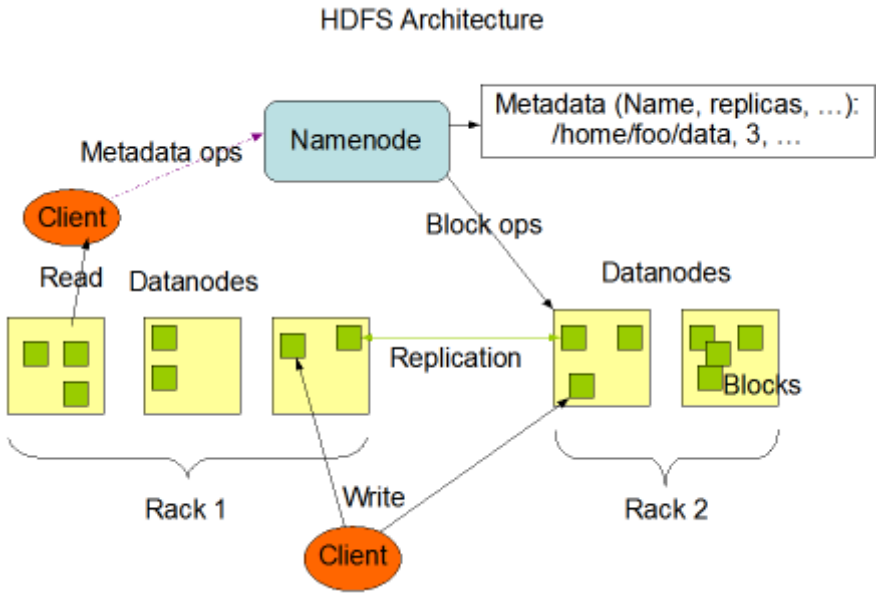
* Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, “The Google File System”, SOSP 2003

分布式文件系统

- **Hadoop分布式文件系统（HDFS）**

  - **简介**
    - Hadoop分布式文件系统，主要由Yahoo!开发的开源系统，面向流数据模式访问和处理超大文件的需求开发，因具有高容错性而可以部署在低廉的硬件上

  - **系统架构** *（主从模式）*
    - 名称节点 (Namenode)：管理文件系统命名空间、客户端对文件的访问操作、数据块与数据节点的映射
    - 数据节点 (Datanode)：管理文件分成的数据块存放在若干节点、处理客户端的文件读写请求



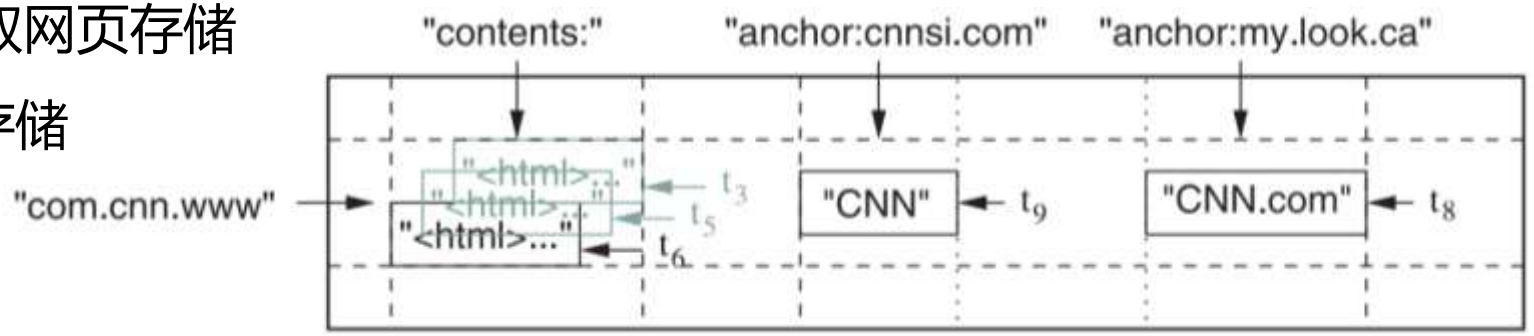
Java语言开发，商用计算机部署集群

Ceph：支持块存储、对象存储和文件存储 *（统一存储）*，支持POSIX接口

分布式数据库

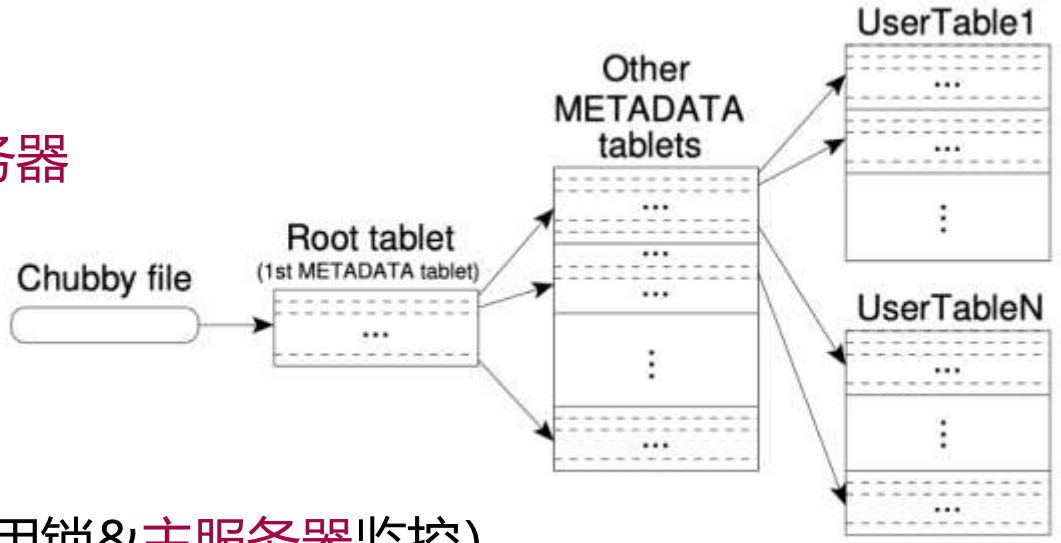
- **Bigtable**

  - **简介**
    - Google开发的管理大规模结构化数据 (PB级) 的分布式KV数据库，开启NoSQL
    - 应用：从后端大批量处理到实时在线服务，包括网页索引、Google分析、Google金融、Orkut、Google Earth、Writely (Google Docs) 等
  - **数据模型**
    - 稀疏的多维排序映射表：(row: string, column: string, time: int64) -> string
    - 支持时间戳多版本 -> 爬取网页存储
    - 底层实际使用key-value存储
    - 数据内容主要是字符串



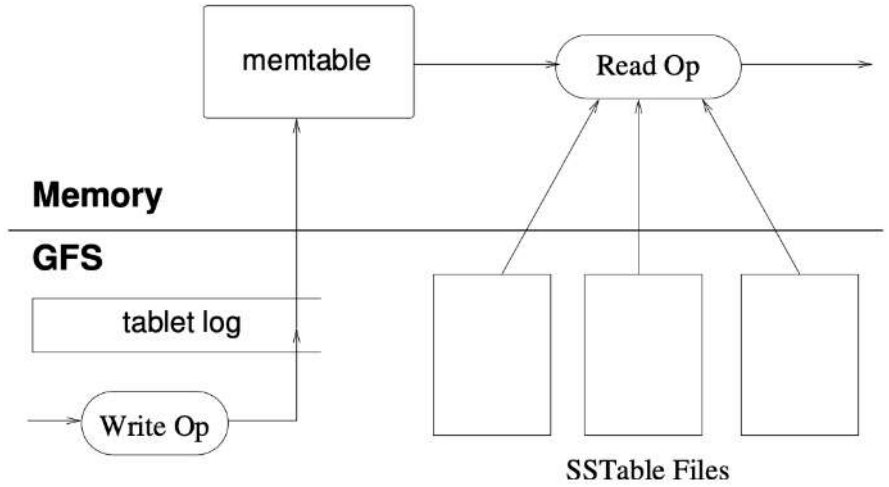
分布式数据库

- **Bigtable**
  - **系统依赖框架**
    - 利用GFS做底层分布式文件/日志存储，负责数据复制 -> 保证可靠性/可用性
    - 利用Chubby保证多副本一致性，存放模式 (表的列簇) 信息和访问控制列表
  - **表块 (Tablet) 分配和定位**
    - 主服务器将表块 (数据分片) 分配到表块服务器
    - 类似B+树的三层结构存放表块位置信息
    - 每个表块服务器管理10-1000个表块
    - 用户直接与表块服务器交互读写表块
    - Chubby目录创建文件跟踪表块服务器（专用锁&主服务器监控）



分布式数据库

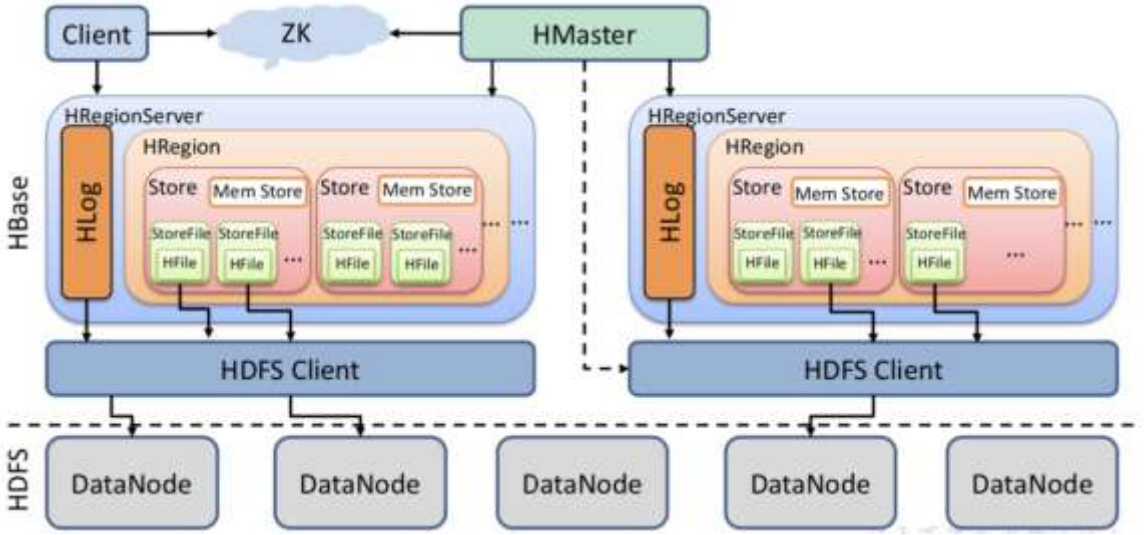
- **Bigtable**
  - **表块服务**
    - 表块永久存放在GFS中，更新记录到提交日志中，最近提交的缓存在memtable
    - 存储格式SSTable，包含序列数据块 (默认64KB)，索引提供从关键字到值的映射
    - 写性能好，读性能较差（相对RocksDB等LSM-Tree存储引擎）
  - **压缩**
    - 小规模 (Minor) 压缩：memtable达到阈值，会创建一个SSTable文件 (内部有序) 写入GFS
    - 主 (Major) 压缩：多个SSTable与memtable合并为一个新SSTable，去掉重复数据加快读速度



分布式数据库

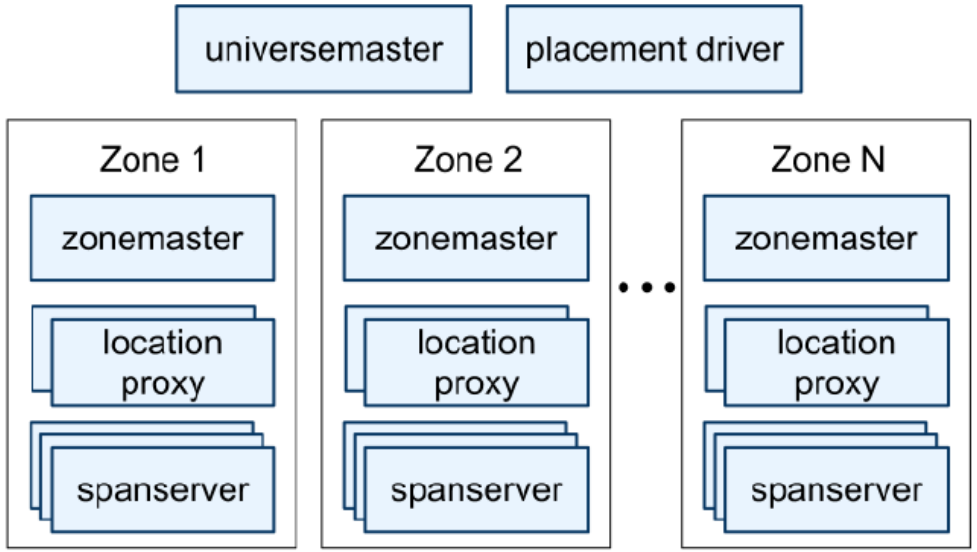
- **HBase**

  - **简介**
    - Apache Hadoop的开源分布式数据库，基于Bigtable开发，面向列的存储模型
    - 存储松散型数据（介于KV和关系型数据库之间），底层依赖于HDFS和ZooKeeper
  - **系统架构** *（主从模式）*
    - HRegion服务器
      - HRegion基于表主键范围分片Region (列簇Store-> MemStore & Hfile)
      - HLOG存储数据日志 (先写模式)
    - HMaster服务器：存储数据分片映射

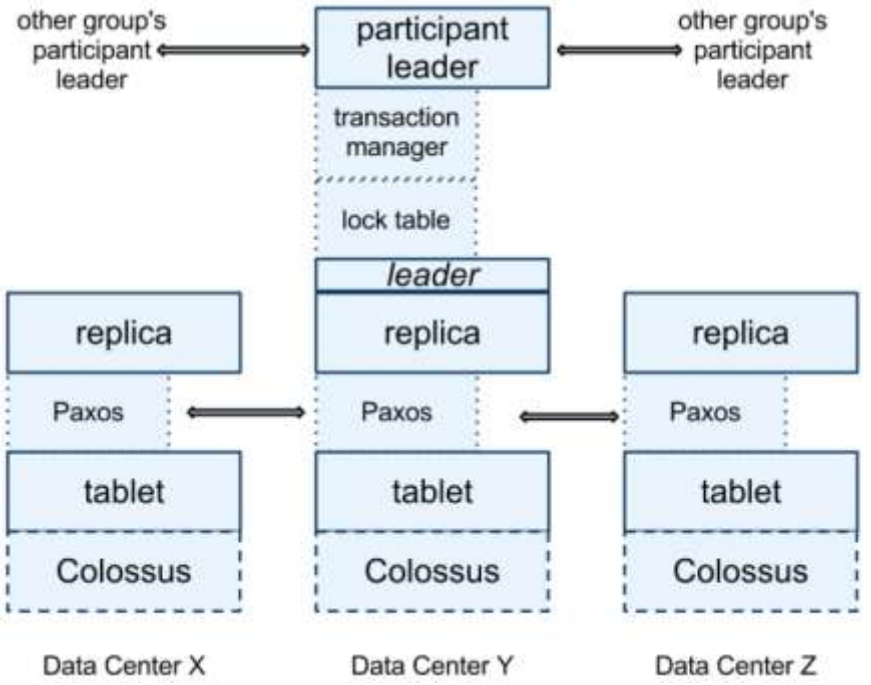


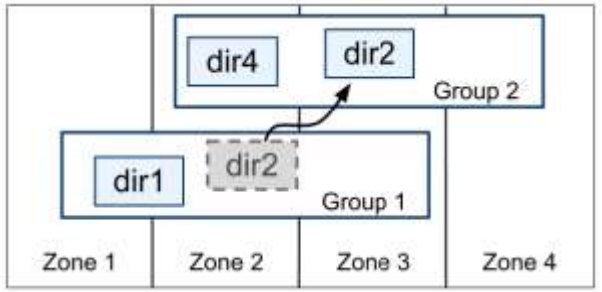
分布式数据库

- **Spanner**
  - **简介**
    - Google开发的全球分布式、多版本存储的KV数据库 *(在Bigtable基础上改进)*
    - 提供多副本同步复制、全局事务特性 (外部一致性语义)、动态无感知后台数据迁移
  - **系统架构**
    - 部署实例 (universe) 有多个管理单元 (zone)
    - 实例包括全局控制器 (显示单元状态) 和放置驱动 (处理单元间的数据迁移)
    - 每个单元包括单元控制器 (分配数据)、工作节点 (处理数据) 和位置代理 (定位工作节点)



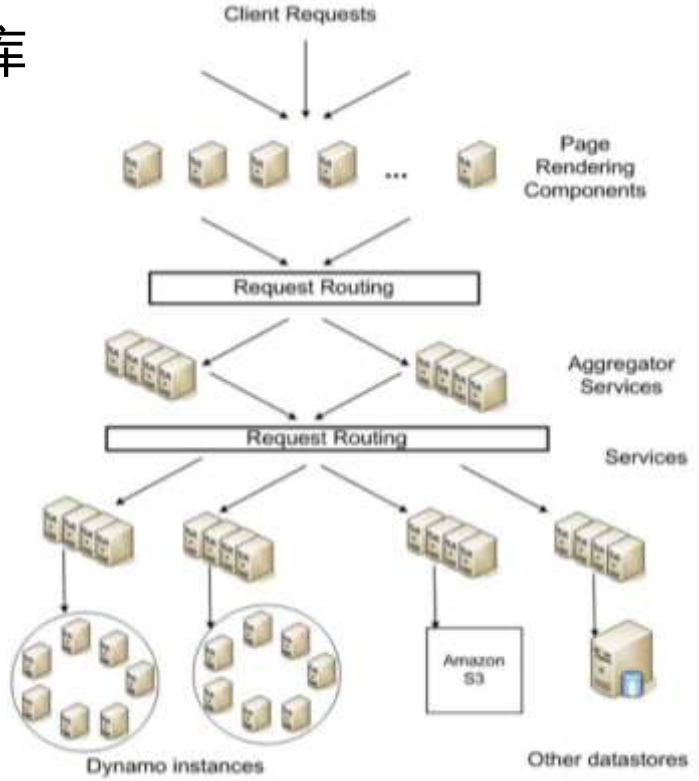
- **Spanner**
  - **工作节点**
    - 提供API (key:string, timestamp:int64)->string
    - 最小管理单位表块 (Tablet) 由多行(k, t)->v组成，存储在类GFS的全局文件系统Colossus上
    - 每个表块都通过Paxos协议在多个单元复制备份，选择其中的一个工作节点作为领导者 *(Paxos组)*
    - 领导者负责并发 (通过锁) 和分布式事务管理
  - **目录和数据迁移**
    - 目录 (dir) 包含一组连续的键，数据迁移基本单位
    - 减轻某个Paxos组的负载压力，或使离访问者更近





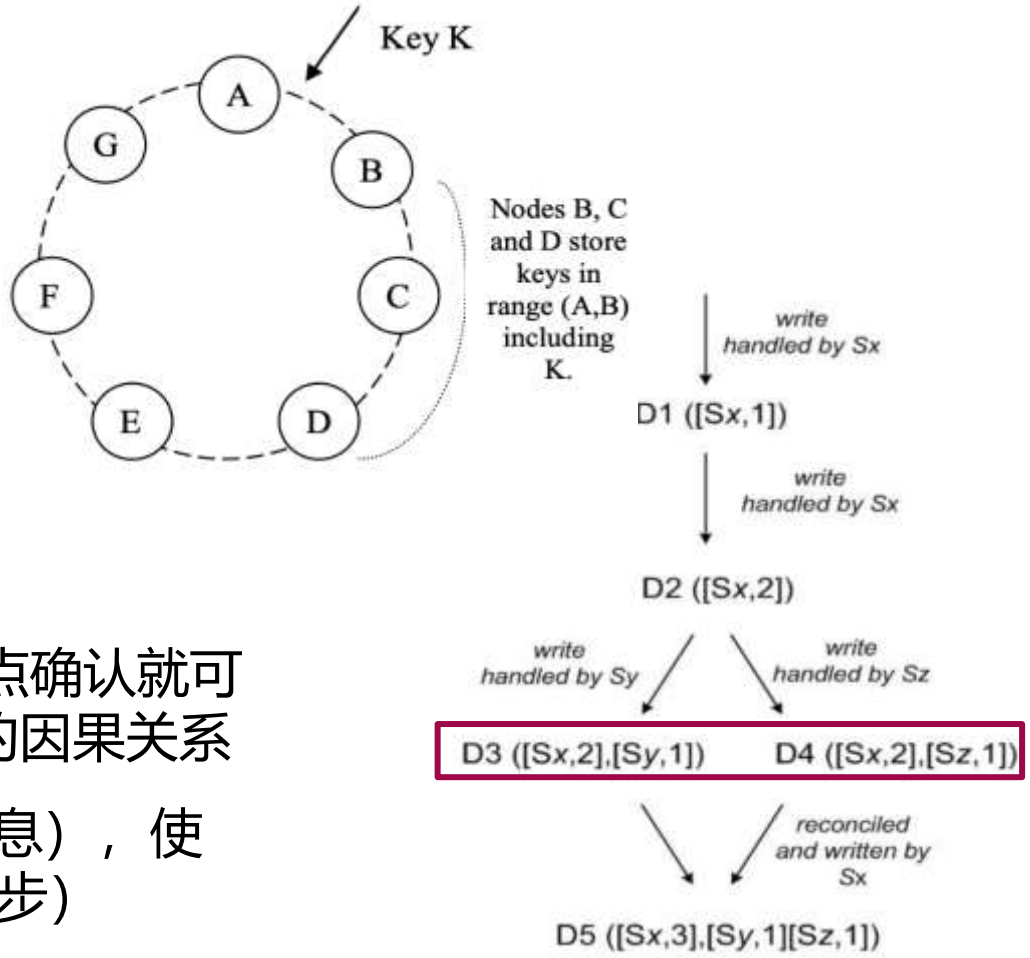
分布式数据库

- **Dynamo**
  - **简介**
    - Amazon开发的高可用分布式KV数据存储，管理松散服务器运行电商平台
    - Cassandra、Voldemort是Dynamo风格的开源分布式数据库
  - **用户访问流程**
    - Dynamo的节点是P2P结构，任何一个节点均可以服务用户
    - 负载均衡器将用户请求（get/put(key, ...)）发送给目标key的协调节点（键的优先列表中第一个无故障节点）
    - 此后由协调节点负责，按照Quorum的R+W>N原则进行读/写访问



分布式数据库

- **Dynamo**
  - **数据分片和复制**
    - 满足规模可扩展性 -> 一致性哈希算法动态确定存取数据的节点，映射到环上的虚拟节点
    - 该虚拟节点（*映射到物理上的协调节点*）本地和顺时针后续N-1个 虚拟节点都存一个副本
  - **版本更新和故障处理**
    - R+W>N不能保证副本同步更新，未经过多数节点确认就可以写操作覆盖 -> 基于向量时钟找出不同版本间的因果关系
    - 通过Gossip协议检测可达节点（避免存储节点信息），使用Merkle树的同步复制来处理故障（后台快速同步）

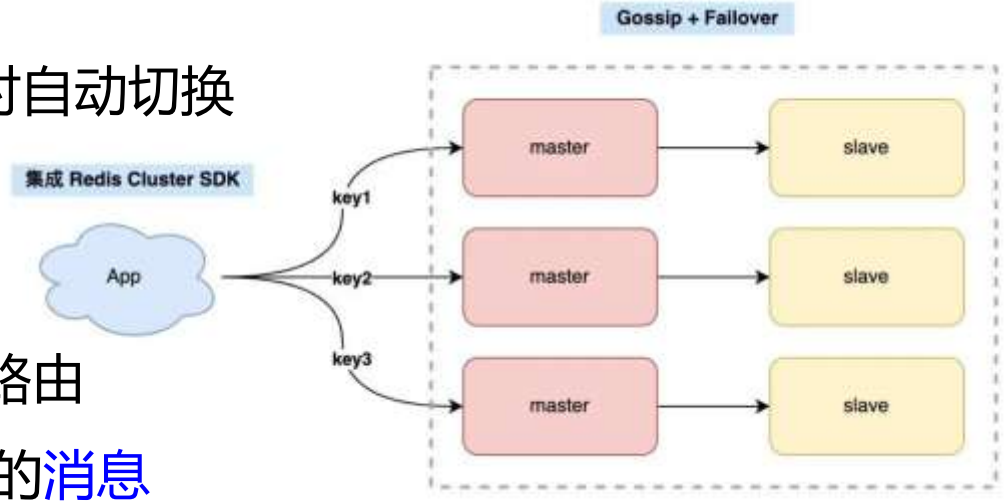


分布式数据库

- **Redis**

  - **简介**
    - 远程字典服务，开源的分布式非关系型 (KV) 数据库 (2009 Merzia->Pivotal)
    - 特点：运行在内存 (读写性能高) & 可持久化到磁盘、对复杂数据结构 (list, set, zset, hash) 提供原子操作、支持主从模式数据备份、支持发布/订阅

  - **系统架构**
    - Redis集群：节点间Gossip协议探测状态，故障时自动切换
    - 客户端通过SDK集成，基于键查找节点读写
    - 自动适配节点增加和删除，业务侧无感知
    - 适配早期业务侧：一组代理负责客户端数据转发路由
    - Redis v5.0新增Stream数据结构，用于消息队列的消息



本课目录

- 分布式系统概述
- 分布式通信
- 分布式存储
- **分布式计算**

分布式计算

- **计算模式**
  - **服务** *（在线系统）*
    - 等待客户请求到达，每收到一个立即处理返回响应，响应时间和可用性是主要指标
  - **批处理** *（离线系统）*
    - 大量输入数据通过一个作业处理，需要一段时间生成输出数据，通常定期运行，不会有用户等待作业完成，吞吐量是主要性能指标 *（MapReduce）*
    - 采用列存储、数据分区与压缩、增量写入等优化技术，使CPU从内存读写数据 *（Spark）*



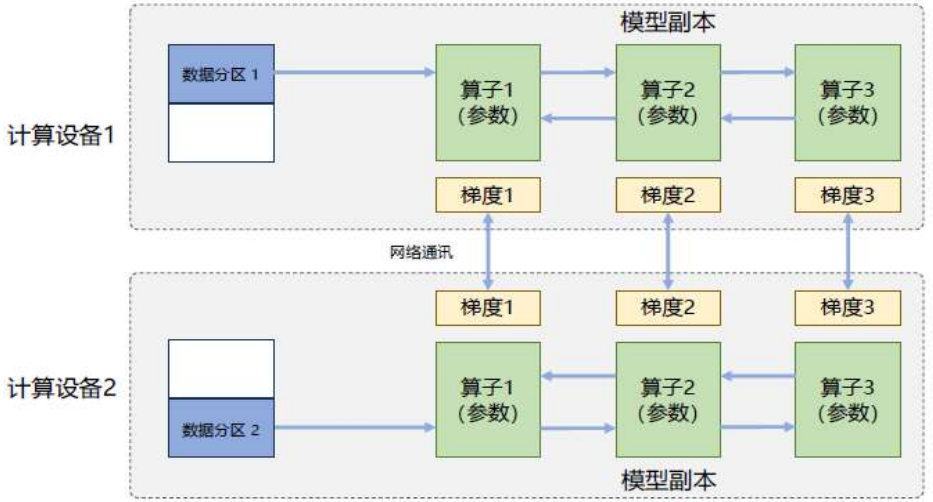
  - **流计算** *（准实时系统）*
    - 介于在线服务和离线批处理之间，被称为准实时或准在线 (nearline) 处理
    - 像批处理一样消费输入并产生输出 (并不响应请求)，但在事件发生后不久就进行操作，比批处理具有更低延迟 *（Flink）*

- **分布式机器学习**
  - **基本思路**
    - 将训练任务分布式地部署到多个计算节点，提高模型训练速度，减少任务耗时
    - 每个计算节点每轮处理样本数量为批尺寸/节点数，各自基于所分配的训练数据对模型进行更新，然后将不同计算节点更新后的模型汇总（每轮结束后在所有节点间同步）
  - **性能优化方法**
    - 降低通信需求：模型尺寸或参数量、控制参数同步频率分别减少需要通信数据量和通信次数，另可调整并行方式和参数同步架构
    - 提升通信能力：利用高性能传输协议实现高带宽、低时延传输，或采用高带宽的网络互联拓扑
    - 提高通信效率：采用通信库定制优化、网内聚合逐条汇聚参数信息，以及采用小尺寸梯度聚合、优先传输紧急参数等通信调度

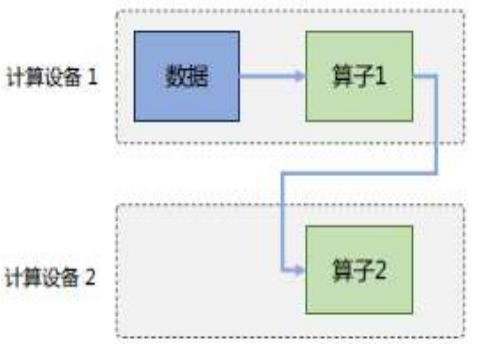
分布式机器学习

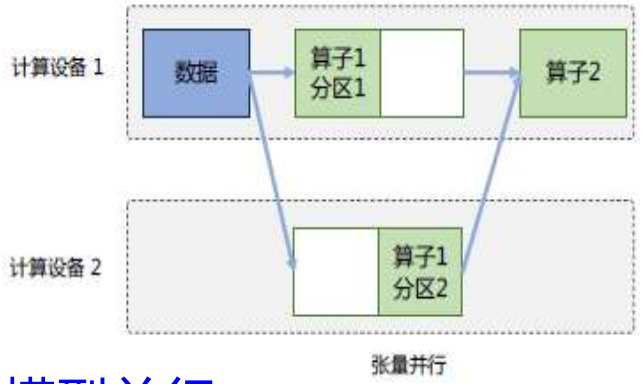
- **并行方式**
  - **数据并行（DP）**
    - 每个计算节点均具有同一模型的副本，但各自分配的训练数据不同，节点间需要将各自的模型更新同步，以保证模型的全局一致性
  - **模型并行（MP）**
    - 将模型划分为多个子模型，并分别部署在不同计算节点，训练数据统一输入，前一节点完成子模型计算将结果传给后一节点

当参数数据量较小时选择数据并行，
当激活值数据量较小时选择模型并行



数据并行

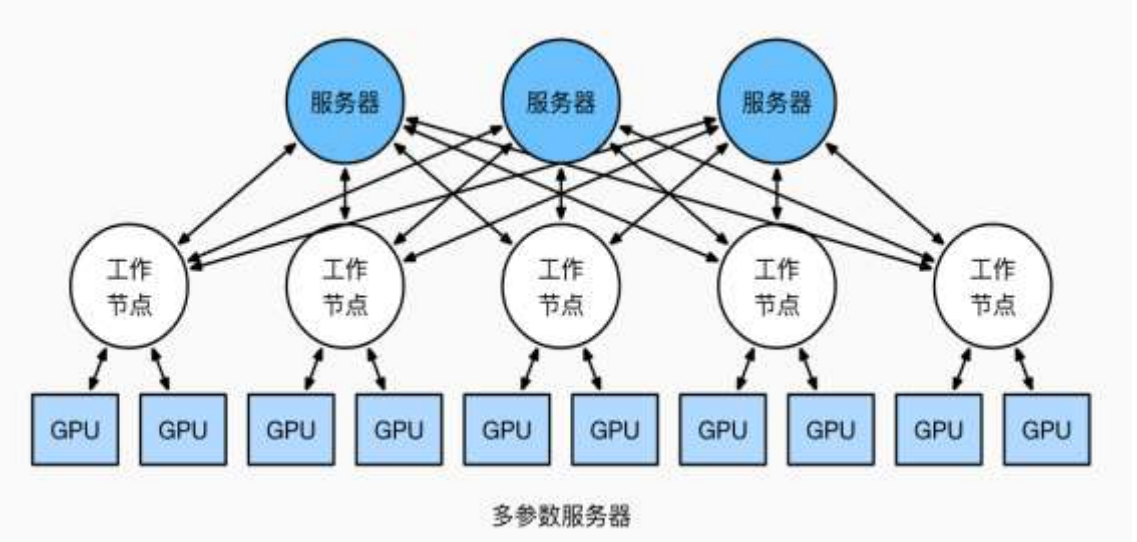




模型并行

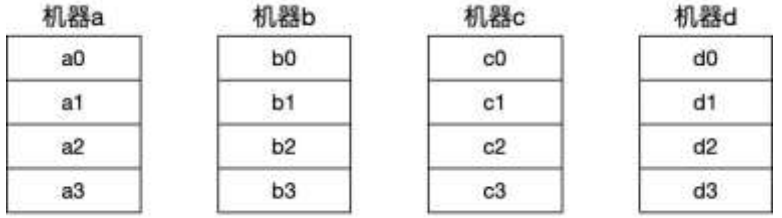
细分为张量并行（TP）& 流水线并行（PP）

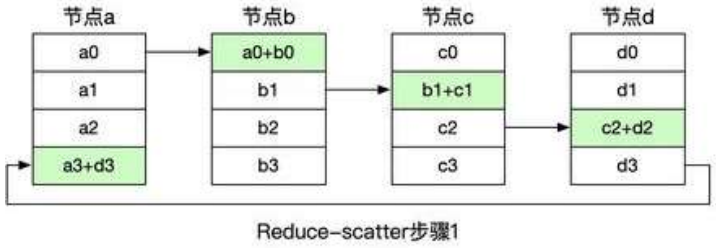
- **参数同步架构**
  - **参数服务器（PS）架构**
    - 服务器节点：保存模型参数，接受工作节点计算出的局部梯度，更新模型参数
    - 工作节点：保存部分训练数据，初始化模型，从服务器节点拉取最新的模型参数，再读取参数，根据训练数据计算局部梯度，上传给服务器节点
    - 同步过程：参数拉取 -> 本地计算 -> 梯度推送 -> 汇总更新
    - 优点：支持各种模型一致性协议、弹性扩展好、鲁棒性强、容错性好
    - 缺点：参数服务器侧易成为网络瓶颈，导致整体训练速度下降



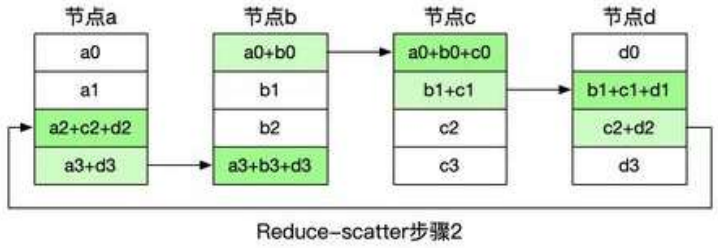
多参数服务器

- **参数同步架构**
  - **环规约**（Ring All-Reduce）**架构**
    - 所有计算节点以逻辑环的形式连接，每个节点只与前后两个邻居节点通信 *(Horovod)*
    - 同步过程：本地计算 -> 分散规约 *(Scatter reduce)* -> 全收集 *(All gather)* -> 参数更新

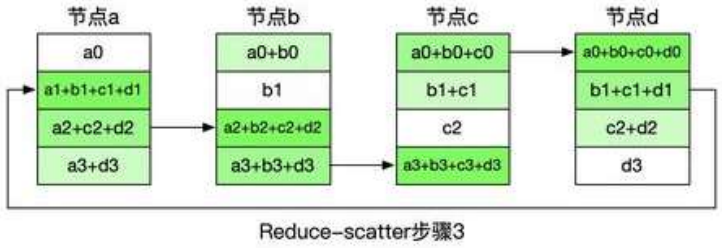




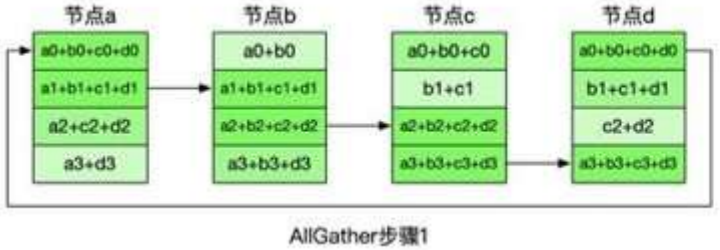
Reduce-scatter步骤1



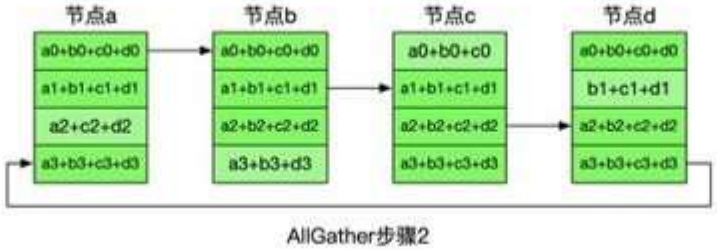
Reduce-scatter步骤2



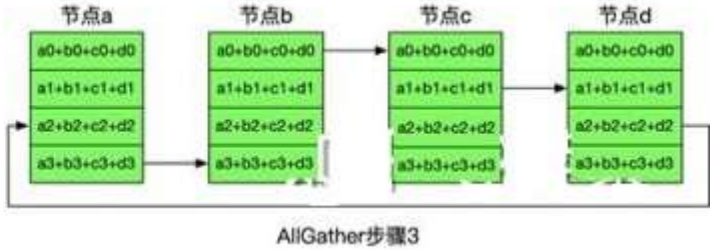
Reduce-scatter步骤3



AllGather步骤1



AllGather步骤2

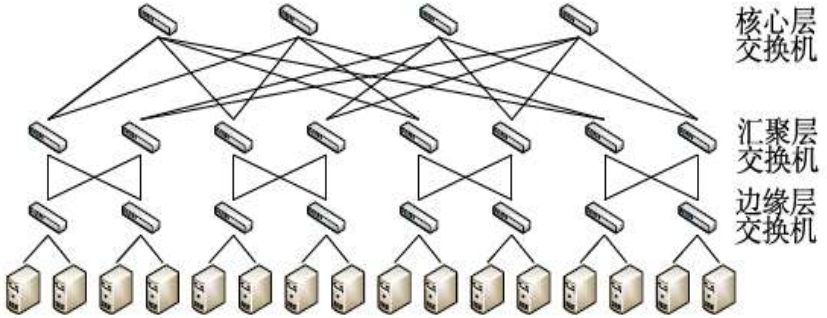


AllGather步骤3

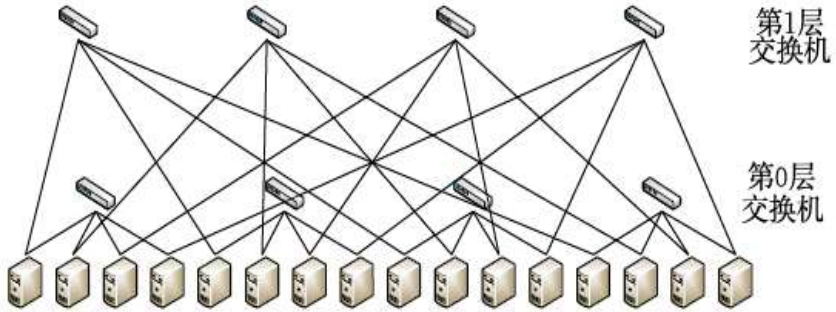
节点数量为n，则进行n-1次分散规约，每次两个节点间传输1/n节点数据量，再进行n-1次全收集

分布式机器学习

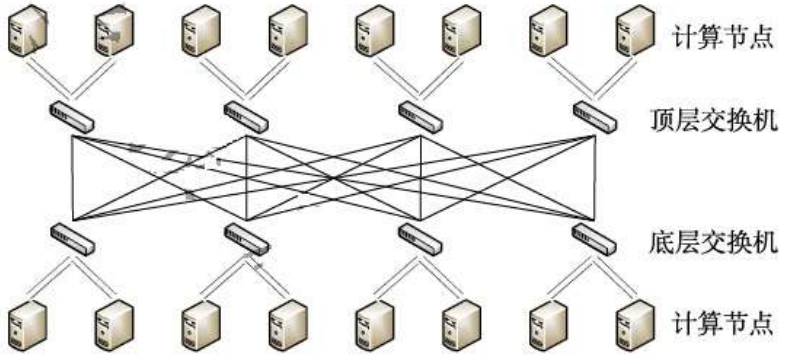
- **网络拓扑**
  - **Fat Tree**：传统数据中心拓扑结构，同步参数时流量负载不均衡
  - **BCube**：以服务器为中心，每个服务器节点都用多个网卡提高互联带宽
  - **BiGraph**（阿里训练集群EFLOPS）：网络分为上下两层通过Clos架构互连，各服务器的每个GPU配备专用网卡，因此服务器与同一交换机可存在多条链路，两层服务器通信最多3跳



Fat Tree



BCube

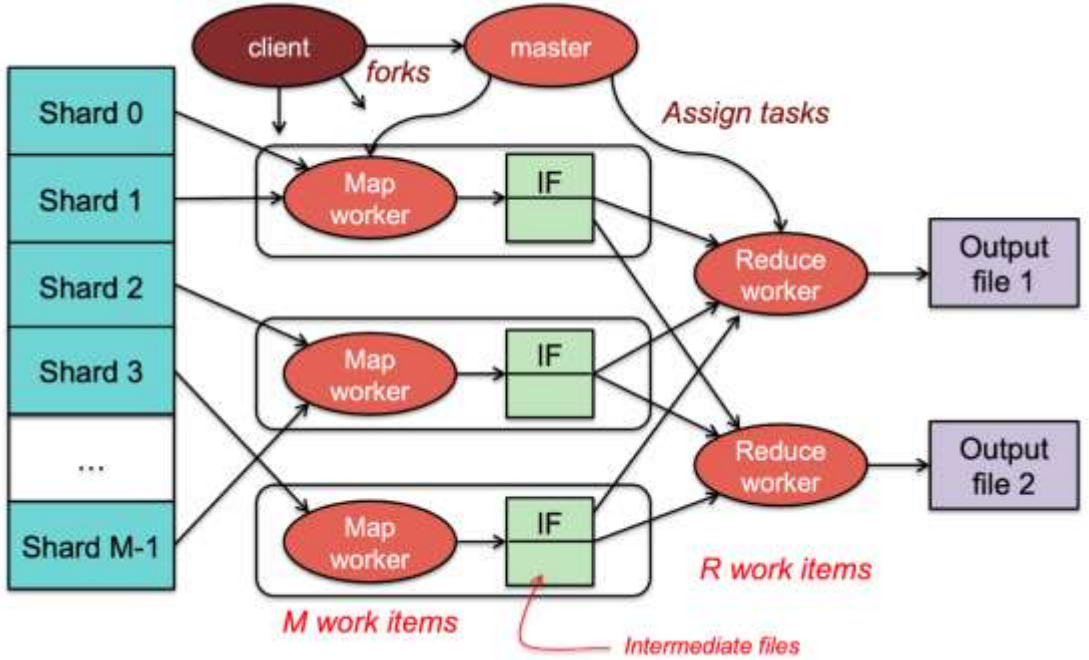


BiGraph

- **传输协议和通信库**
  - **DCTCP**：针对数据中心场景对TCP优化，交换机加ECN标记反馈拥塞，调节发送窗口
  - **RDMA**：无需CPU干预直接访问远程主机内存 *（零内存拷贝、绕过内核、不占CPU时间）*
    - 具体标准：InfiniBand (IB) *（专用网络协议栈）*、RoCE、iWARP *（以太网&专用网卡）*
    - 机器学习通信：IB Verbs *（GPU-内存-内存-GPU）*、GDR *（GPU显存-GPU显存）*
  - **NVLink**：Nvidia推出的总线和协议，在多GPU间以及GPU与CPU间实现高速互联
    - P100 (2016) 1.0 160GB/s -> V100 (2017) 2.0 300GB/s -> A100 (2020) 3.0 600GB/s
    - NCCL（集合通信库）：allgather、 reduce、broadcast等集合通信原语
    - NCCL兼容MPI，1.0支持NVLink、PCIe高速通信，2.0支持RDMA、TCP多机多卡通信

经典计算框架

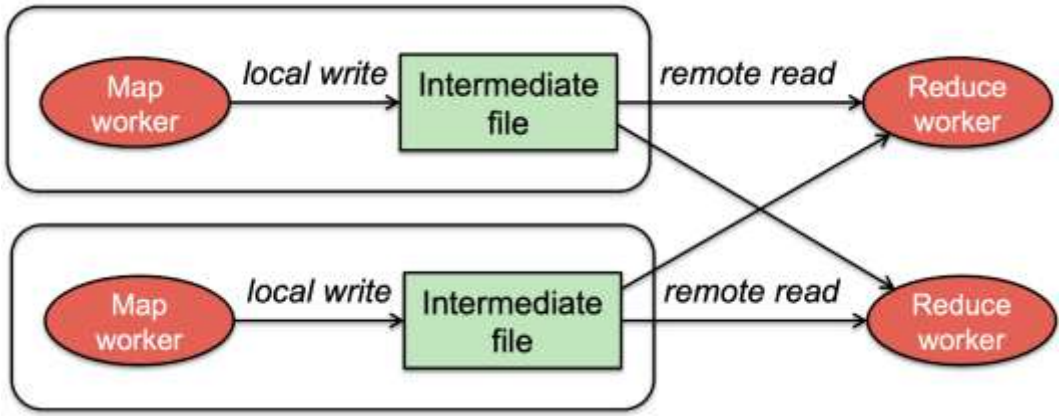
- **MapReduce**
  - **简介**
    - 一种用于大规模数据集（一般大于1TB）的并行运算编程模型 *（大数据处理框架）*
    - 应用场景：Google网页处理建立索引， Hadoop大规模企业数据处理和分析
  - **模型框架**
    - Map函数：从数据源（如HDFS）获取键值对<key, value>集合 -> 拆分数据分片输入计算，生成一个中间<key, value>集合
    - Reduce函数：接收一个中间key和所有相关value集合 -> 将合并结果生成一个新的输出<key, value>集合



- **MapReduce**
  - **执行步骤**
    - 拆分：将输入数据分成M个分片，一般大小为64MB
    - 派生（Forking）：管理者向空闲工作者分配Map任务或Reduce任务
    - Map：将分片解析出键值对，输入函数计算中间键值对，周期性写入中间文件
    - 分区：将任务划分给R个工作者，将Map结果计算键的哈希值和它们匹配

      hash(key) mod R
    - 洗牌（Shuffling）：读取中间文件数据，按键排序并分组相同键的数据
    - Reduce：将key和对应的一组中间值输入函数，计算后追加到输出文件

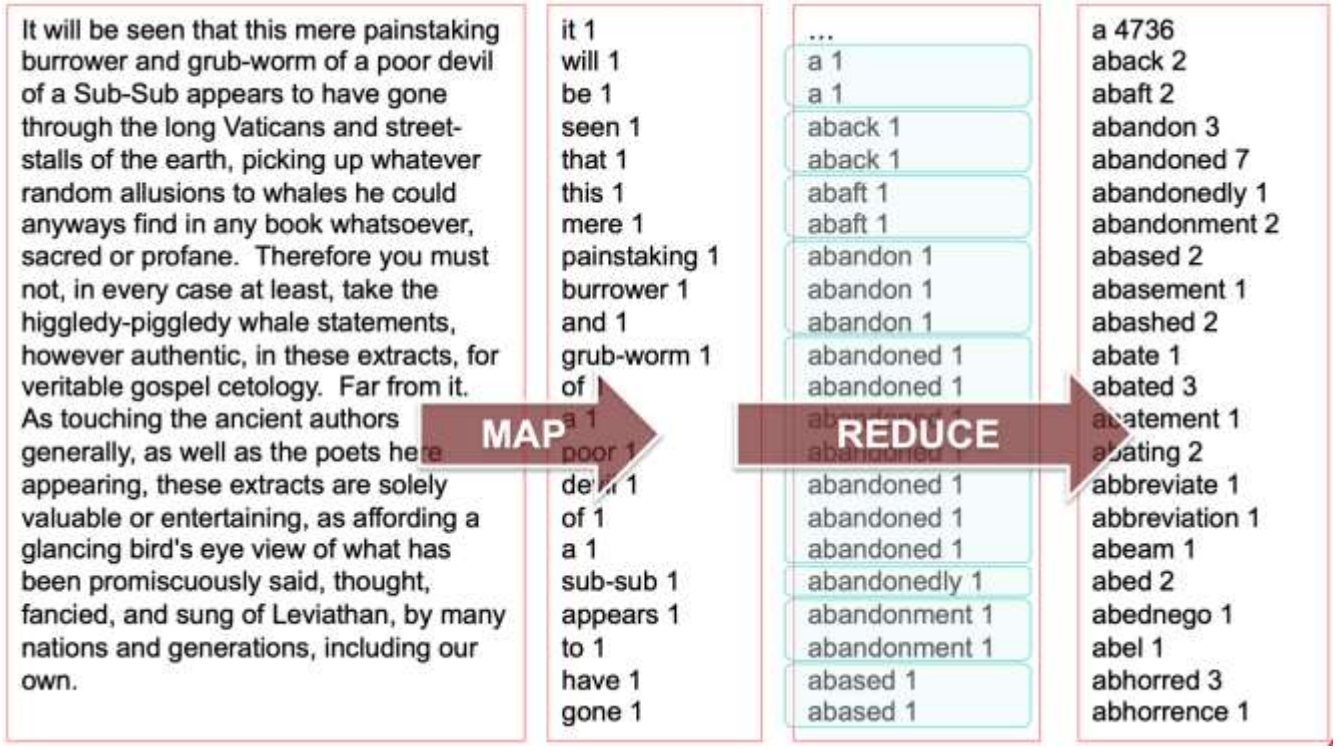
      <key, (value1, value2, value3, value4, ...)>



- **MapReduce**
  - **典型实例：单词计数**

```
Map(String key, String value):
    for each word w in value:
        EmitIntermediate(w, '1');

Reduce(String key, Iterator values):
    int result = 0;
    for each v in values:
        Result += ParseInt(v);
    Emit(AsString(result));
```

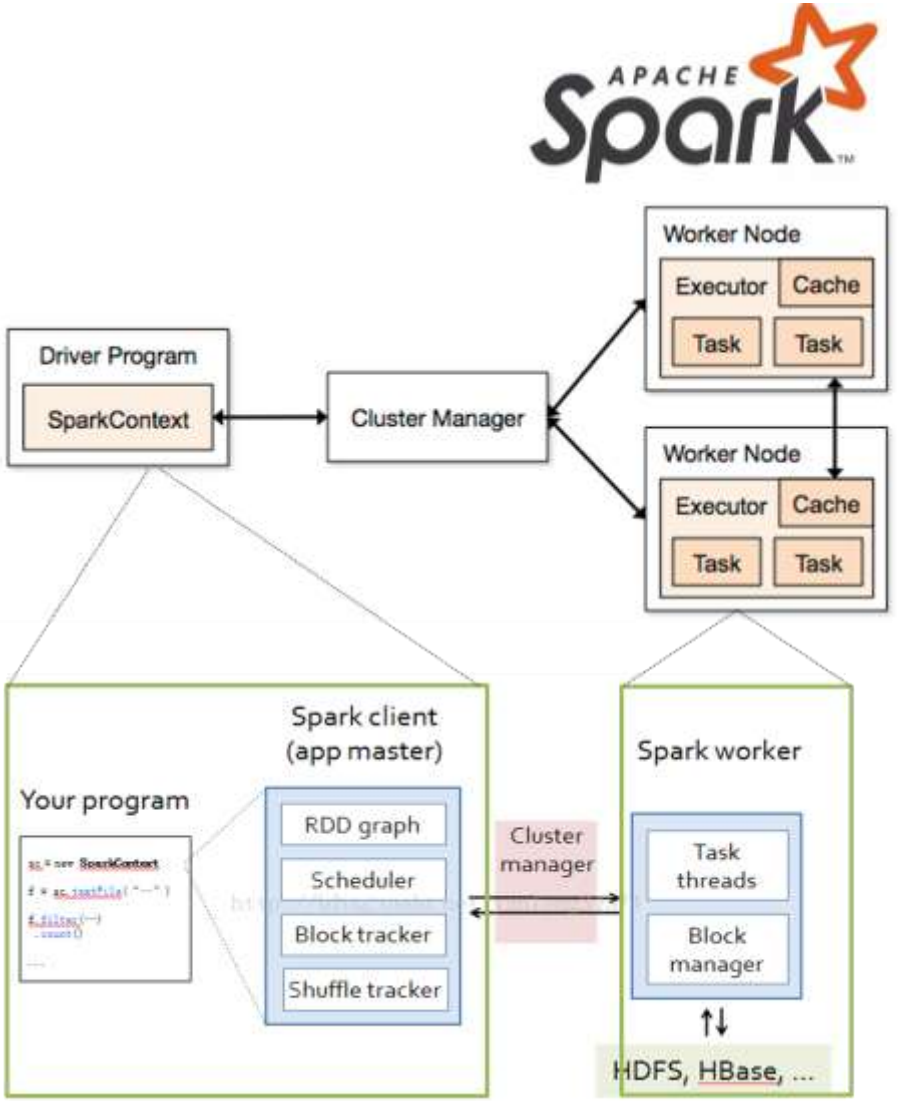


输入参数	Map函数	Reduce函数
key	文档名称	一个单词
value	文档内容	计数列表

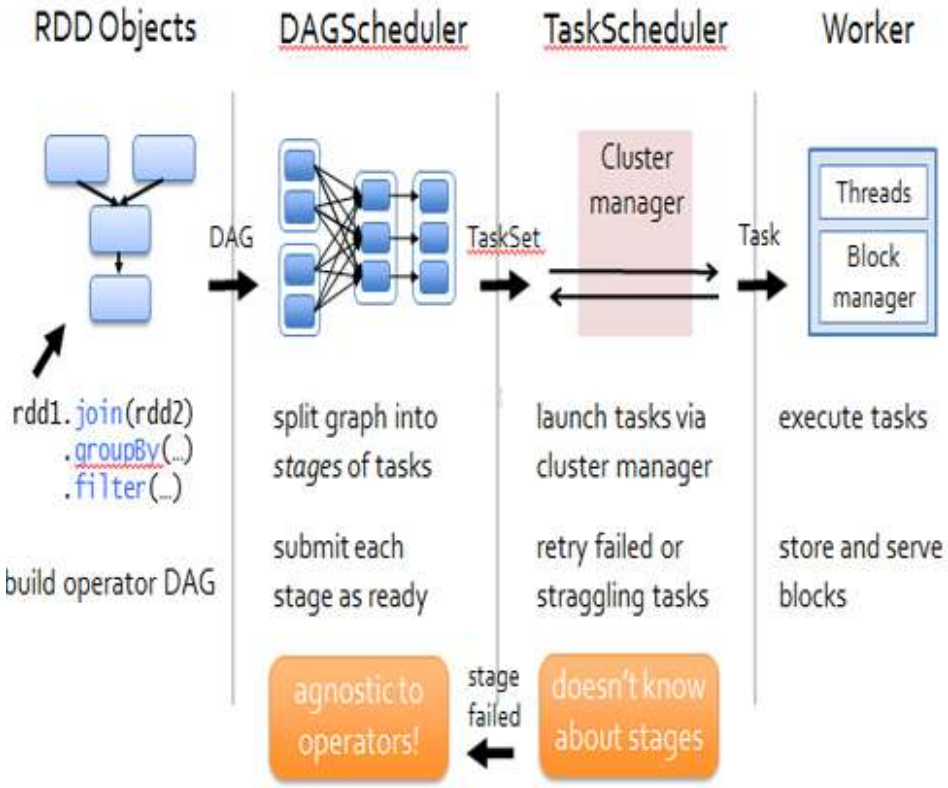
- **内存计算Spark**

  - **简介**
    - UC Berkeley开源的大规模数据分析一体化引擎，类似MapReduce但使用内存计算的通用并行框架
    - 缓存机制支持反复迭代计算或者多次数据共享

  - **工作流程**
    - 驱动器向集群管理器 (YARN/Mesos) 申请运行任务所需资源，为任务分配工作节点并创建执行器
    - 驱动器将用户编写的作业代码拆分成多个阶段，每个阶段创建一批任务，分配到各执行器执行
    - 执行器在阶段全部任务执行后将中间结果返回驱动器或写入HDFS，驱动器再进行下一阶段调度



- **内存计算Spark**
  - **调度过程**
    - 体现不同于MapReduce的DAG思想
    - DAG调度器把一个Spark作业转换成阶段DAG，按阶段以任务集的形式提交给任务调度器
    - 任务调度器在集群的执行器运行具体任务，重试失败或滞后的任务，并与底层资源调度系统交互
    - 弹性分布式数据集（RDD）：一种容错并行数据结构，允许用户保留中间结果，控制分片来优化在内存中的位置信息

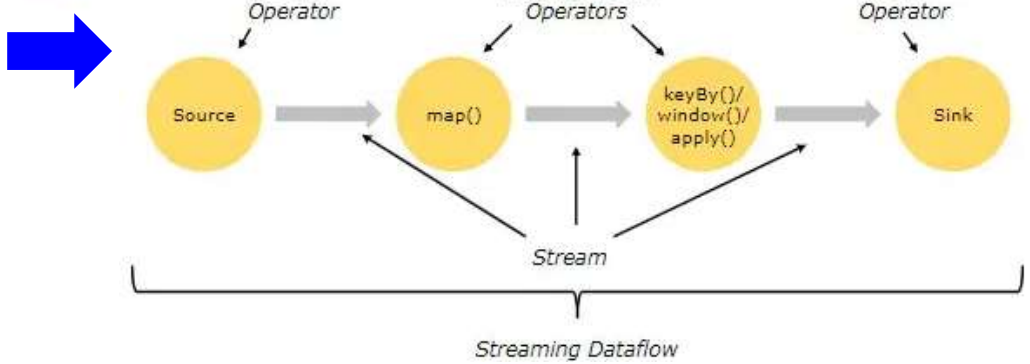


经典计算框架

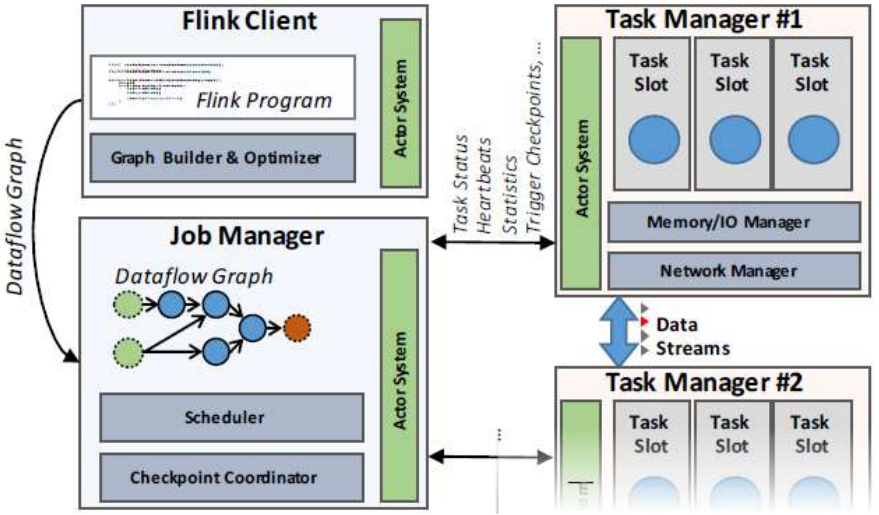
- **流计算Flink**



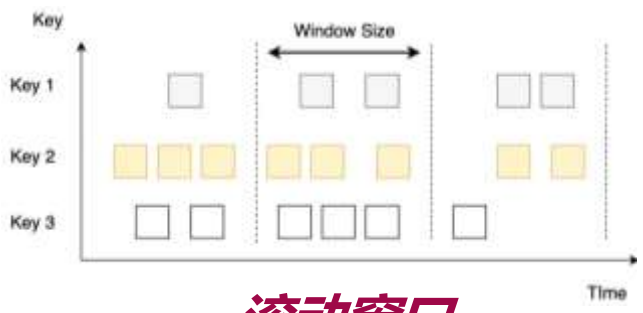




  - **简介**
    - 诞生于TU Berlin的大数据研究项目StratoSphere，2014孵化出Flink并捐赠给Apache开源->Apache顶级大数据项目
    - 阿里于2018年底收购德国公司Data Artisans
  - **系统架构**
    - 客户端：接收代码翻译成数据流图，查询优化
    - 作业管理器：调度数据流，跟踪流和算子的状态，协调检查点和数据流恢复
    - 任务管理器：执行算子产生流，维护网络连接 (基于HTTP/gRPC) 交换数据



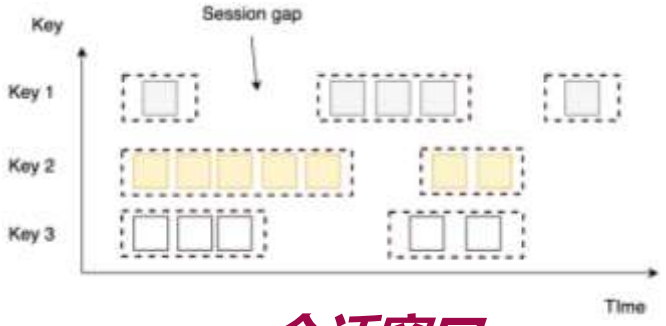
- **流计算Flink**
  - **窗口机制**
    - 数据流是无限的 -> 为了计算方便划分为窗口 (Window)
    - 滚动窗口：窗口之间不重叠，且窗口长度固定
    - 滑动窗口：窗口以步长 (Slide) 不断向前滑动，窗口长度固定
    - 会话窗口：窗口在会话间隔时间内没接收到新数据则关闭切分，窗口长度可变



滚动窗口



滑动窗口



会话窗口

经典计算框架

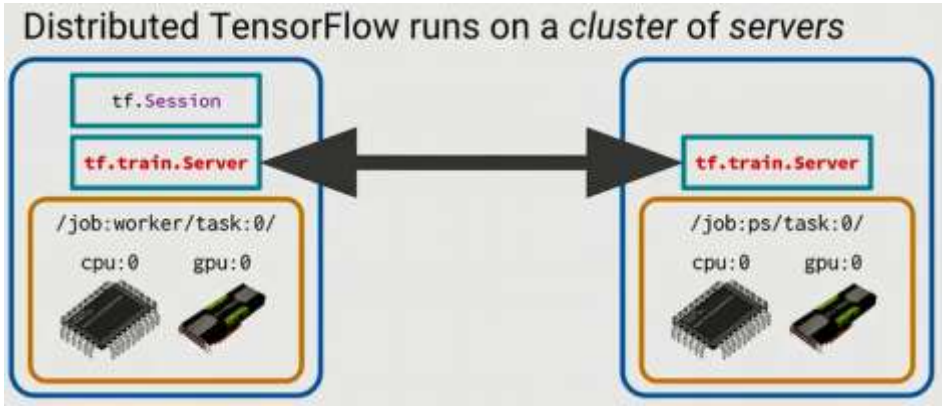
- **TensorFlow**

TensorFlow

  - **简介**
    - Google Brain开发的开源机器学习框架，致力于数据流图的自动微分和DNN计算
    - Horovod：Uber于2017年发布的一个基于TensorFlow的高性能分布式深度学习训练框架，对常见的NCCL等优化库做了打包，提高了易用性

HOROVOD

  - **系统架构**
    - Cluster：分布式系统的所有节点或者设备（Server的集合）
    - Server：与Task对应，包含Master和Worker组件
    - Client：一个程序，创建了计算图，与Cluster交互

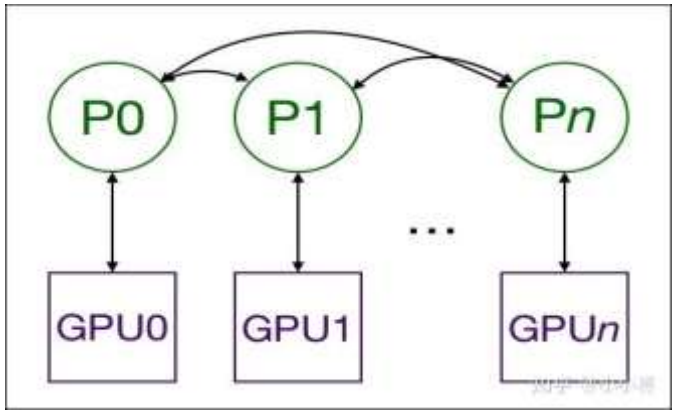


经典计算框架

- **PyTorch**

PyTorch

  - **简介**
    - Facebook于2016年发布的开源深度学习框架，在主流框架中对于灵活性和易用性的平衡最好
  - **训练模式**
    - 两种方法在多个GPU上切分模型和数据
    - DataParallel：参数服务器 (PS) 架构，单进程多线程，在每个批处理期间将分发到每个GPU
    - DistributedDataParallel：环规约 (Ring All-Reduce) 架构，支持模型并行，多进程，每个进程有自己的Python解释器和优化器



推荐阅读论文

- **分布式共识**
  - 1. **[SIGACT News'01]** Leslie Lamport, “Paxos Made Simple” *(Paxos简述)*
  - 2. **[TOCS'98]** Leslie Lamport, “The Part-Time Parliament” *(Paxos)*
  - 3. **[ATC'14]** Diego Ongaro and John Ousterhout, “In Search of an Understandable Consensus Algorithm” (Extended Version) *(Raft)*
  - 4. **[ATC'10]** Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed, “ZooKeeper: Wait-free coordination for Internet-scale systems” *(ZooKeeper)*
  - 5. **[LADIS'08]** Benjamin Reed, and Flavio P. Junqueira, “A simple totally ordered broadcast protocol” *(ZAB)*
  - 6. **[OSDI'06]** Mike Burrows, “The Chubby lock service for loosely-coupled distributed systems” *(Chubby)*
  - 7. **[TPLS'82]** Leslie Lamport, Robert Shostak, Marshall Pease. “The Byzantine Generals Problem” *(拜占庭将军问题)*

推荐阅读论文

- **分布式系统**
  - 1. **[MSST'10]** Konstantin Shvachko, Hairong Kuang, Sanjay Radia, Robert Chansler, "The Hadoop Distributed File System" *(HDFS)*
  - 2. **[OSDI'06]** Sage A. Weil, Scott A. Brandt, Ethan L. Miller, et al, "Ceph: A Scalable, High-Performance Distributed File System" *(Ceph)*
  - 3. **[OSDI'12]** James C. Corbett, Jeffrey Dean, Michael Epstein, et al, "Spanner: Google's Globally-Distributed Database" *(Spanner)*
  - 4. **[SOSP'07]** Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al, "Dynamo: Amazon's Highly Available Key-value Store" *(Dynamo)*
  - 5. **[NSDI'12]** Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing" *(Spark RDD)*
  - 6. **[IEEE DEB'15]** Paris Carbone, Stephan Ewen, Seif Haridi, "Apache Flink™: Stream and Batch Processing in a Single Engine" *(Flink)*
  - 7. **[NetDB'11]** Jay Kreps, Neha Narkhebe, and Jun Rao, "Kafka: a Distributed Messaging System for Log Processing" *(Kafka)*

推荐阅读论文

- **分布式机器学习**
  - 1. **[OSDI'16]** Martín Abadi , Ashish Agarwal , Paul Barham, et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems" *(TensorFlow)*
  - 2. **[arXiv'18]** Alexander Sergeev, Mike Del Balso, "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow" *(Horovod)*
  - 3. **[NeurIPS'19]** Adam Paszke , Sam Gross , Francisco Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library" *(PyTorch)*
  - 4. **[ICML'22]** Samyam Rajbhandari, Conglong Li, et al., "DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale" *(DeepSpeed)*
  - 5. **[ATC'23]** Xianyan Jia, Le Jiang, Ang Wang, et al., "Whale: Efficient Giant Model Training over Heterogeneous GPUs" *(Whale)*
  - 6. **[ACL'23]** Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, et al., "Petals: Collaborative Inference and Fine-Tuning of Large Models" *(Petals)*
  - 7. **[SIGCOMM'24]** Kun Qian, Yongqing Xi, Jiamin Cao, et al, "Alibaba HPN: A Data Center Network for Large Language Model Training" *(HPN)*

推荐书目和课程

- **推荐书目**
  - Martin Kleppmann著，《数据密集型应用系统设计》，中国电力出版社，2018 *（DDIA，豆瓣评分9.7）*中文翻译 https://github.com/Vonng/ddia



- **推荐课程**
  - MIT 6.824 Distributed Systems (Robert Morris) B站视频 主页课件


RENMIN UNIVERSITY OF CHINA
1937
中国人民大学


RENMIN UNIVERSITY OF CHINA
1937
中国人民大学

谢谢